



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

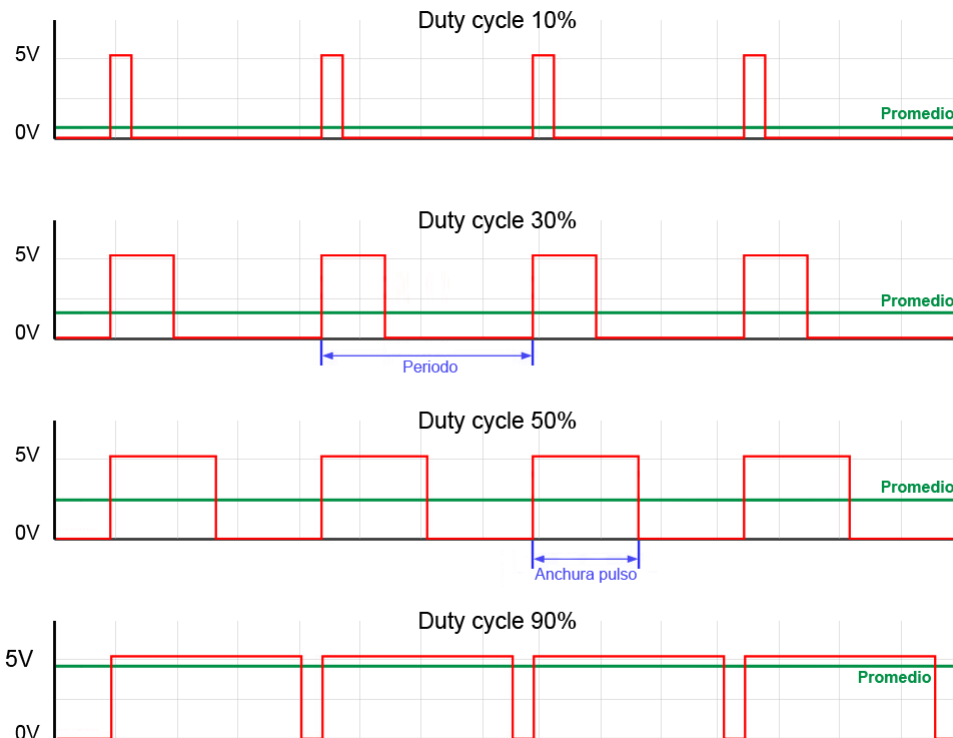
Facultad de Ingeniería
UNLP

PERIFERICOS TIMER
Modos PWM

Ing. José Juárez

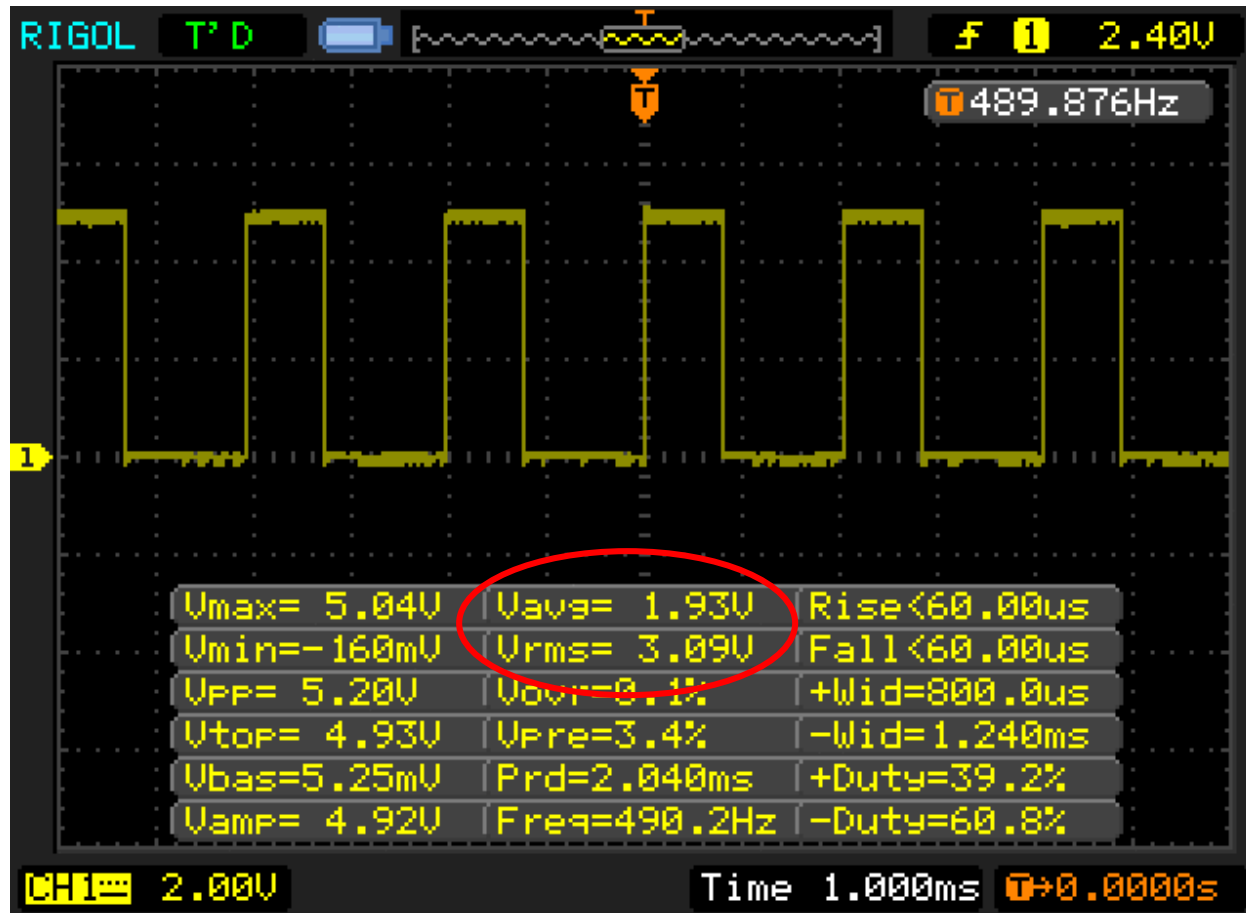
Señales PWM (Pulse Width Modulation)

- Es una técnica de modulación digital donde la información útil de la señal se encuentra en el ancho del pulso.
- Esto permite que a partir de una señal digital de ancho de pulso variable se pueda obtener cierta información en forma analógica.



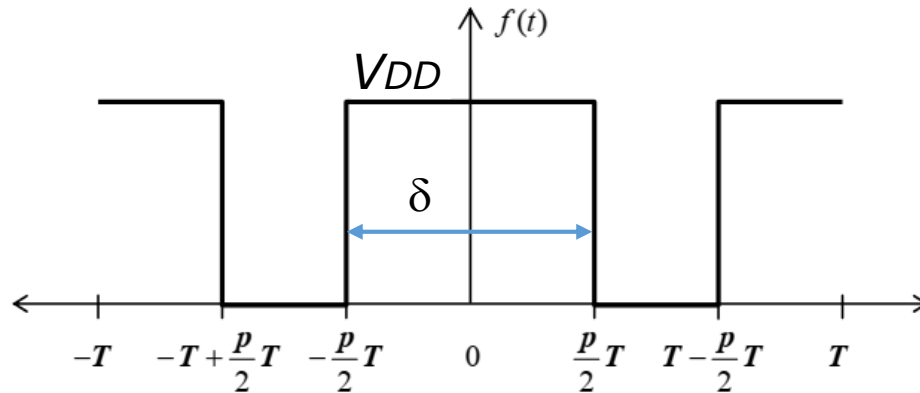
- Es decir, permite controlar dispositivos analógicos por medio de salidas digitales o generar señales analógicas a partir de la conversión digital-analógica (D/A)
- Ejemplos: control de velocidad de motores DC, control de intensidad de lámparas, control de color en LED RGB.

PWM ejemplos



PWM análisis de Fourier

- Sea $f(t)$ una señal PWM de periodo T y ciclo de trabajo $0 \leq p \leq 1$



$$p = \delta/T$$

- La representación por Series de Fourier de la señal es:

$$f(t) = A_0 + \sum_{n=1}^{\infty} \left[A_n \cos\left(\frac{2n\pi t}{T}\right) + B_n \sin\left(\frac{2n\pi t}{T}\right) \right] \quad A_n = \frac{1}{T} \int_{-T}^T f(t) \cos\left(\frac{2n\pi t}{T}\right) dt$$

$$A_0 = \frac{1}{2T} \int_{-T}^T f(t) dt$$

Valor medio o componente DC

$$B_n = \frac{1}{T} \int_{-T}^T f(t) \sin\left(\frac{2n\pi t}{T}\right) dt$$

armónicos

PWM análisis de Fourier

- Si la amplitud de la señal es $V_{OH}=V_{DD}$

$$A_0 = V_{DD} p$$

Información de interés!

El valor medio de la señal es proporcional al ciclo de trabajo p

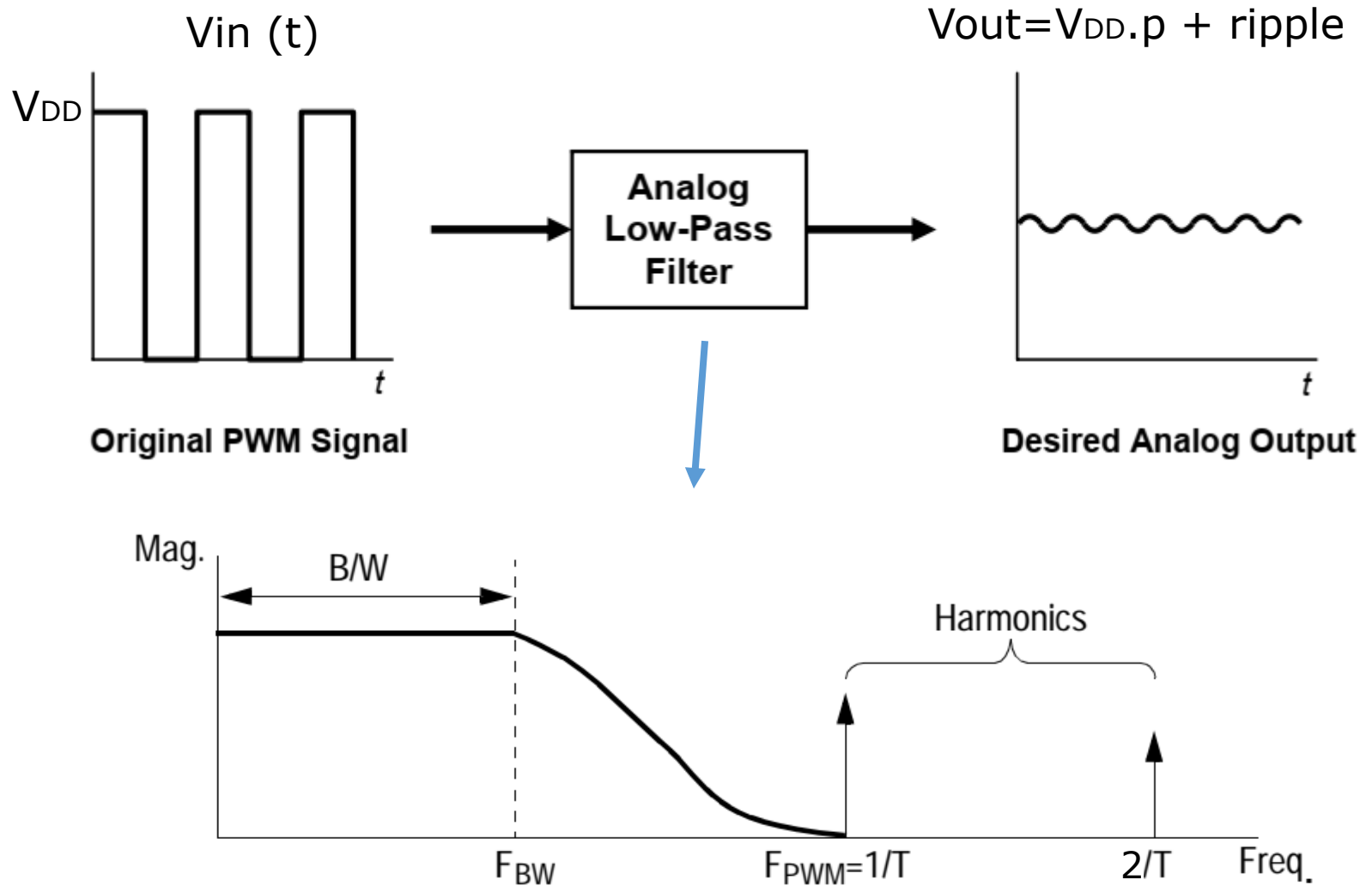
$$A_n = V_{DD} \frac{1}{n\pi} [\sin(n\pi p) - \sin(2n\pi(1 - p/2))]$$

$$B_n = 0$$

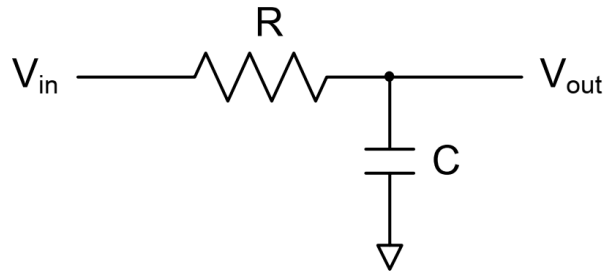
Amplitud de la n -ésima armónica

- La señal PWM contendrá una componente de DC de amplitud A_0 (valor medio de la señal) y las armónicas de la frecuencia PWM a $f=n*f_{PWM}$ (con n entero) y de amplitud A_n respectivamente.
- El valor eficaz de la señal PWM será $V_{rms}=V_{DD} \sqrt{p}$

PWM Filtrado



PWM Filtrado



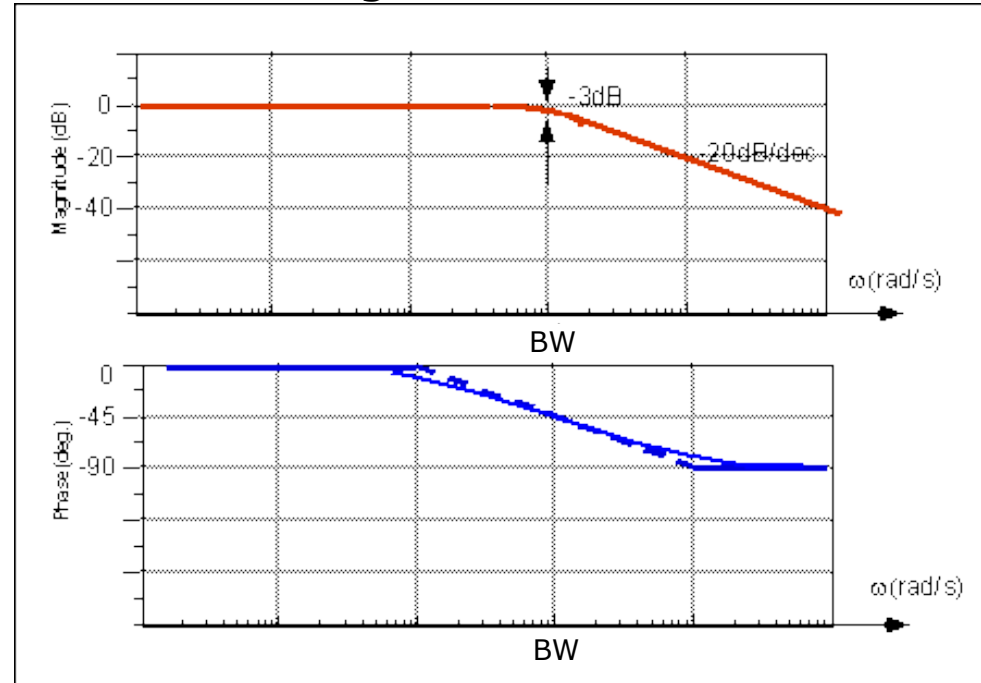
$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{1}{\tau s + 1}$$

$$\tau = RC \quad s = j\omega$$

$$BW = \frac{1}{\tau} \quad (\text{rad/s})$$

$$V_{out}(t) \cong A_0 + A_1 |H(f_{pwm})| \cos(2\pi f_{pwm} t + \phi(f_{pwm}))$$

Diagrama de Bode



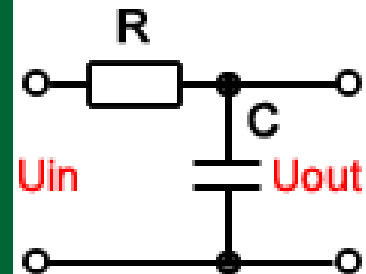
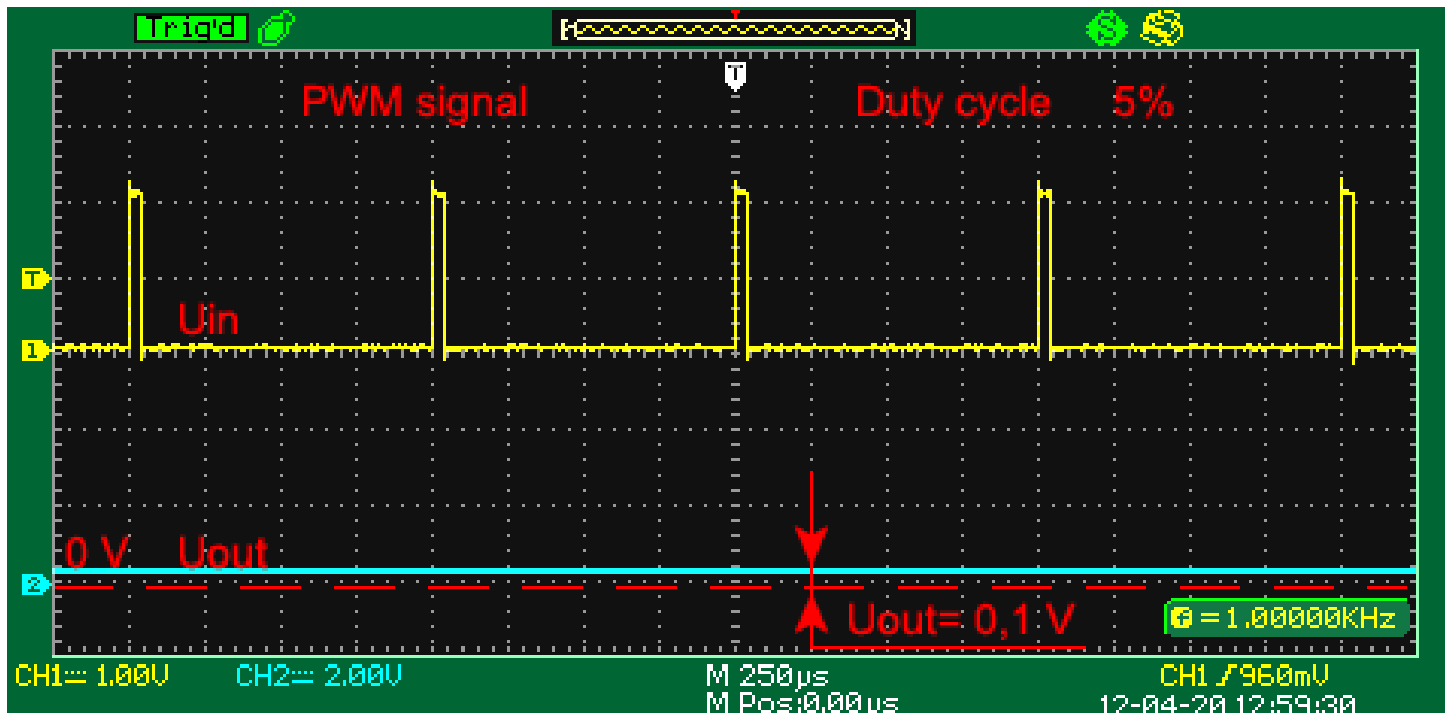
$$dB = 10 \log \left(\frac{V_{out}}{V_{in}} \right)^2 = 10 \log |H(f)|^2$$

$$|H(f)| = \frac{1}{\sqrt{1 + (2\pi \times f \times RC)^2}}$$

$$\phi(H(f)) = -\arctg(2\pi f RC)$$

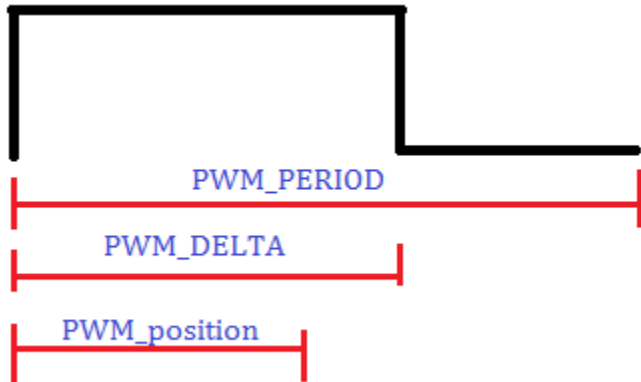
*Si hacemos $BW = \omega_{pwm}/10$
entonces la primera armónica será
atenuada 20dB es decir 10 veces!*

PWM Filtrado



<http://sim.okawa-denshi.jp/en/CRlowkeisan.htm>

Generación de PWM por Software



```
#define PWM_PERIOD 100
#define PWM_DELTA 25
#define PWM_OFF PORTB &=~(1<<PORTB0)
#define PWM_ON PORTB |=(1<<PORTB0)
#define PWM_START DDRB |=(1<<PORTB0)
```

```
// interrupción periódica cada 1ms
```

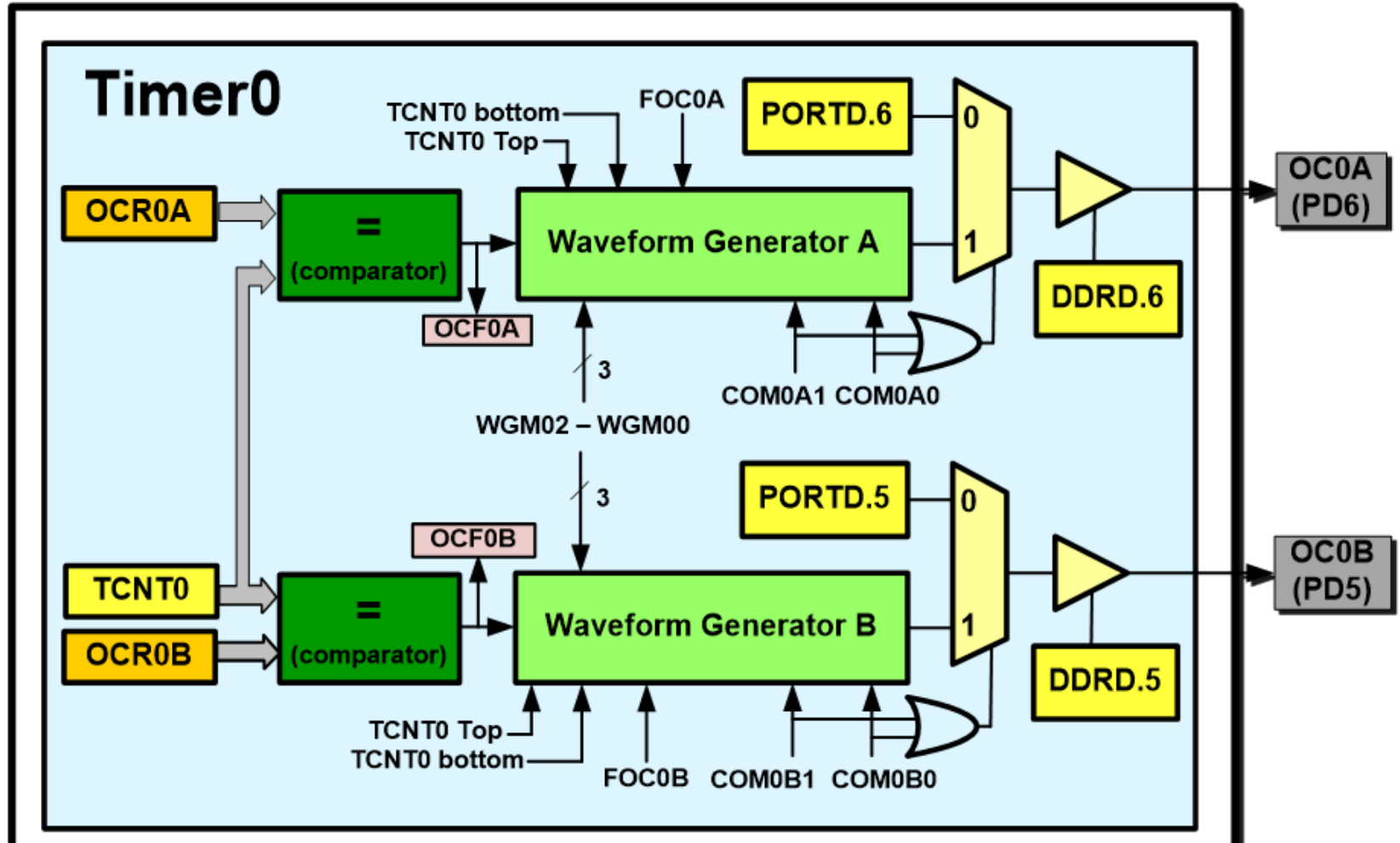
```
ISR (TIMER0_COMPA_vect)
```

```
{
    PWM_soft_Update();
}
```

```
void PWM_soft_Update(void){
    static uint16_t PWM_position=0;
    if (++PWM_position>=PWM_PERIOD){
        PWM_position=0;
    }
    if (PWM_position<PWM_DELTA){
        PWM_ON;
    } else{
        PWM_OFF;
    }
}
```

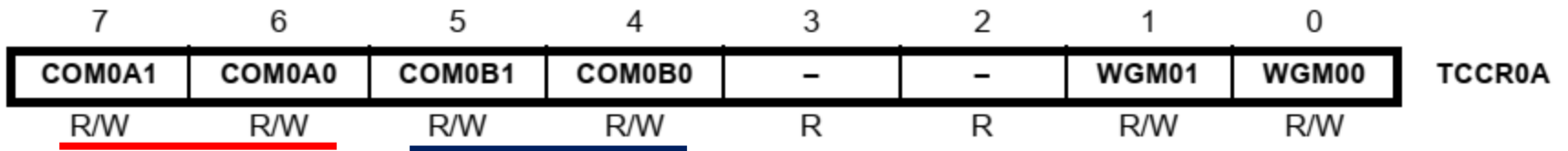
*¿cual es la frecuencia de la señal PWM generada en PB0?
¿cual es el ciclo de trabajo?*

Generación de PWM por Hardware



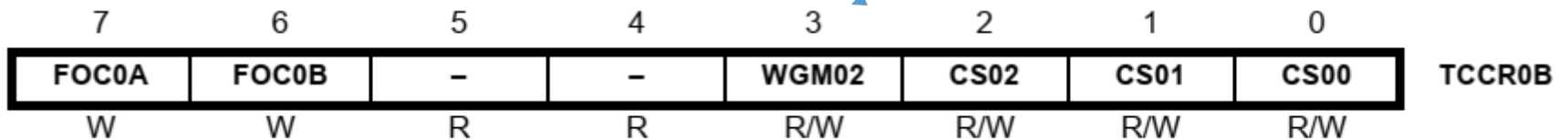
PWM – Timer 0

- Recordemos los registros de configuración TCCR0A y B



Compotamiento de los
generadores de señal **A** y **B**

Modos



Fuente y predivisor
de reloj

PWM – Timer 0

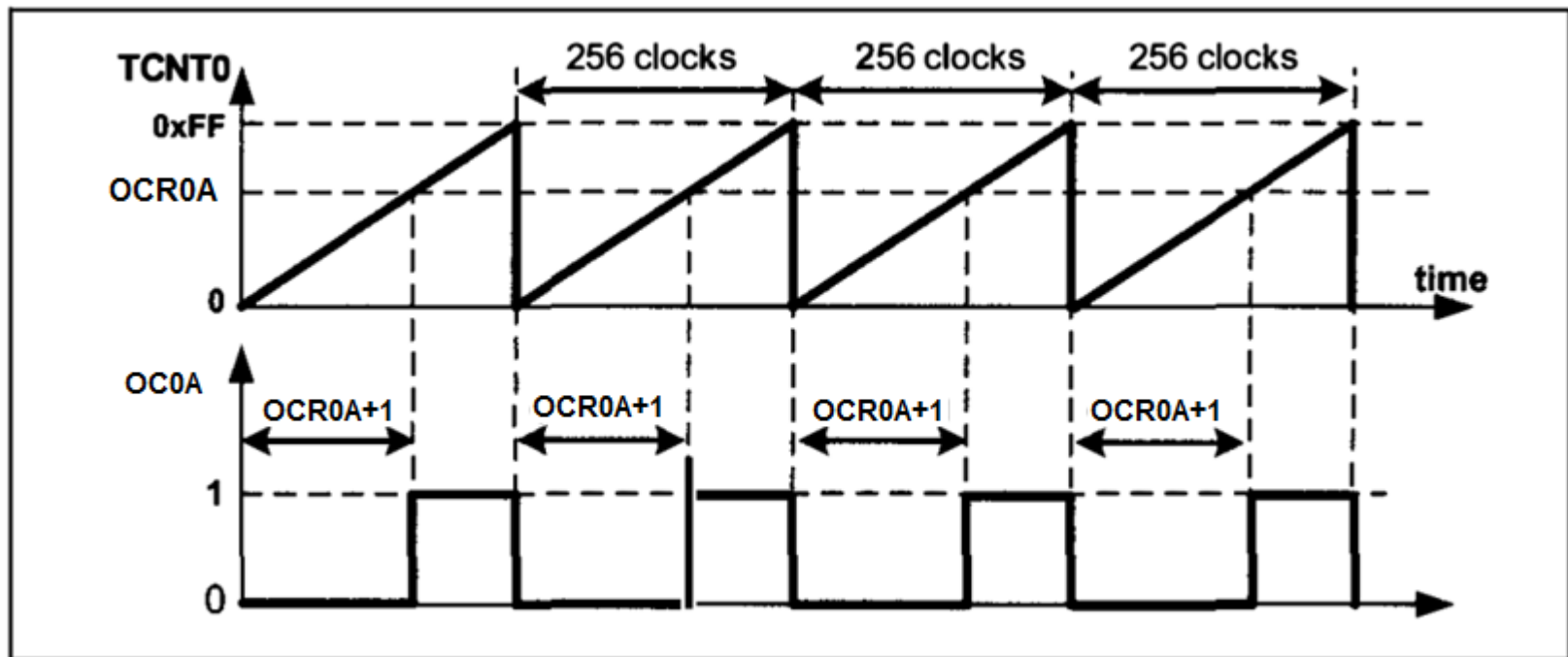
Table 14-8. Waveform Generation Mode Bit Description

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Notes: 1. MAX = 0xFF
2. BOTTOM = 0x00

Generador PWM – Timer 0

- Modo Fast PWM modo 3: operación



Modo Fast PWM Modo Invertido

PWM – Timer 0

Table 14-3. Compare Output Mode, Fast PWM Mode⁽¹⁾

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected. WGM02 = 1: Toggle OC0A on Compare Match.
1	0	Clear OC0A on Compare Match, set OC0A at BOTTOM, (non-inverting mode).
1	1	Set OC0A on Compare Match, clear OC0A at BOTTOM, (inverting mode).

Modo Fast PWM: invertido, no invertido

PWM – Timer 0

- Utilizaremos el generador de onda y la salida de comparación con las siguientes posibilidades:

COM0A1=1
COM0A0=0
no invertido

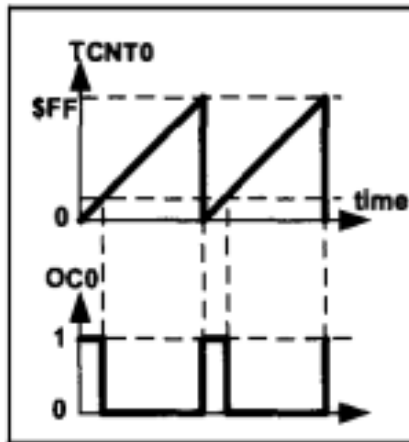


Figure 16-13A. Non-inverted

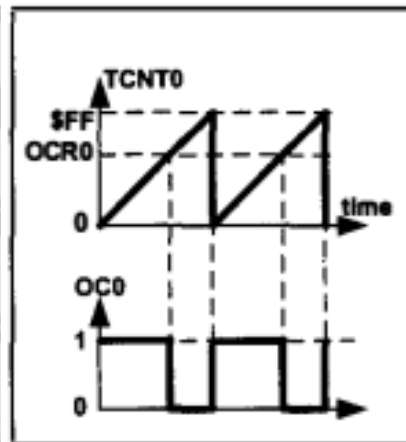


Figure 16-13B. Non-inverted

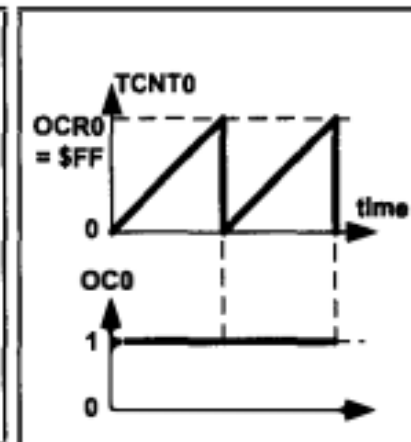


Figure 16-13C. Non-inverted

COM0A1=1
COM0A0=1
invertido

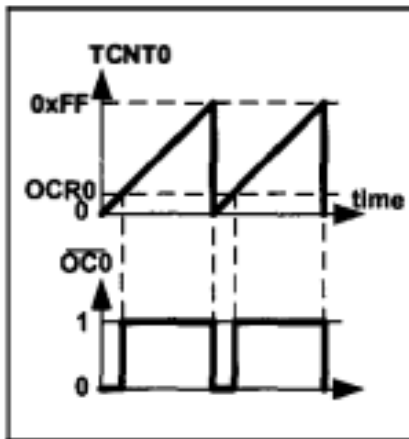


Figure 16-14A. Inverted

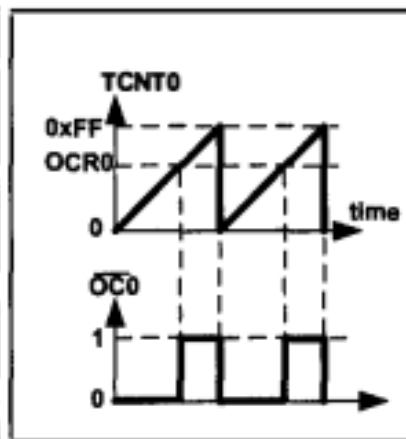


Figure 16-14B. Inverted

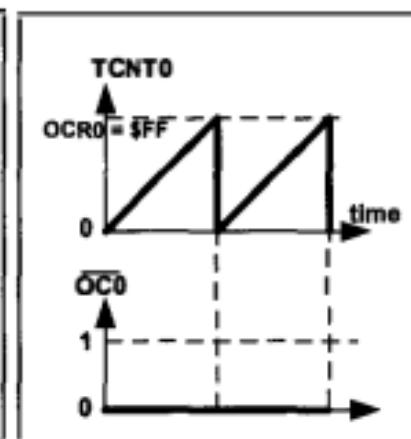
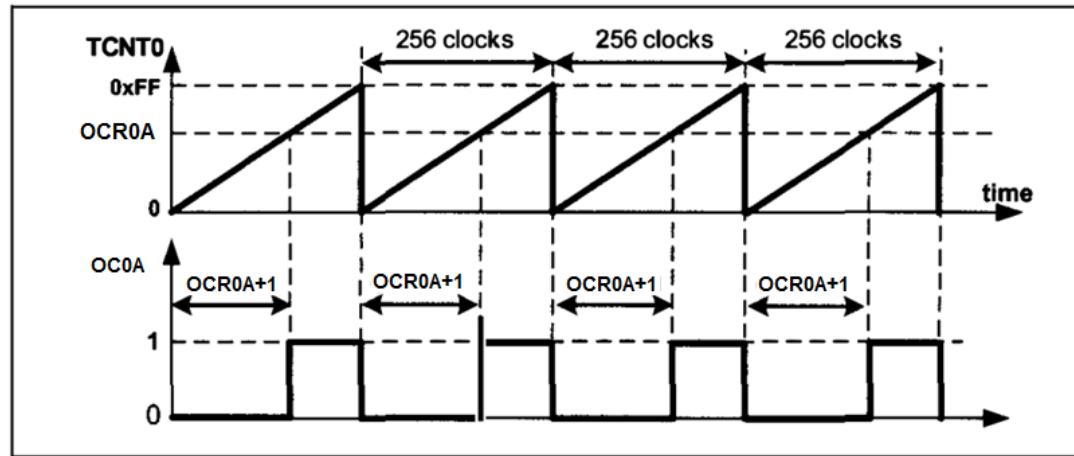


Figure 16-14C. Inverted

PWM – Timer 0



- Frecuencia de la señal generada Modo 3:

$$\left. \begin{aligned} F_{\text{generated wave}} &= \frac{F_{\text{timer clock}}}{256} \\ F_{\text{timer clock}} &= \frac{F_{\text{oscillator}}}{N} \end{aligned} \right\} \Rightarrow F_{\text{generated wave}} = \frac{F_{\text{oscillator}}}{256 \times N}$$

- Ciclo de trabajo: $p = \delta/T$

$$\text{Duty Cycle} = \frac{\text{OCR0} + 1}{256} \times 100$$

Modo no invertido

$$\text{Duty Cycle} = \frac{255 - \text{OCR0}}{256} \times 100$$

Modo invertido

PWM – Timer 0

- Ejemplo: Generar pwm con $f=7812.5\text{Hz}$ y $CT=37.5\%$
 - Supongamos $f_{clk}=16\text{MHz}$

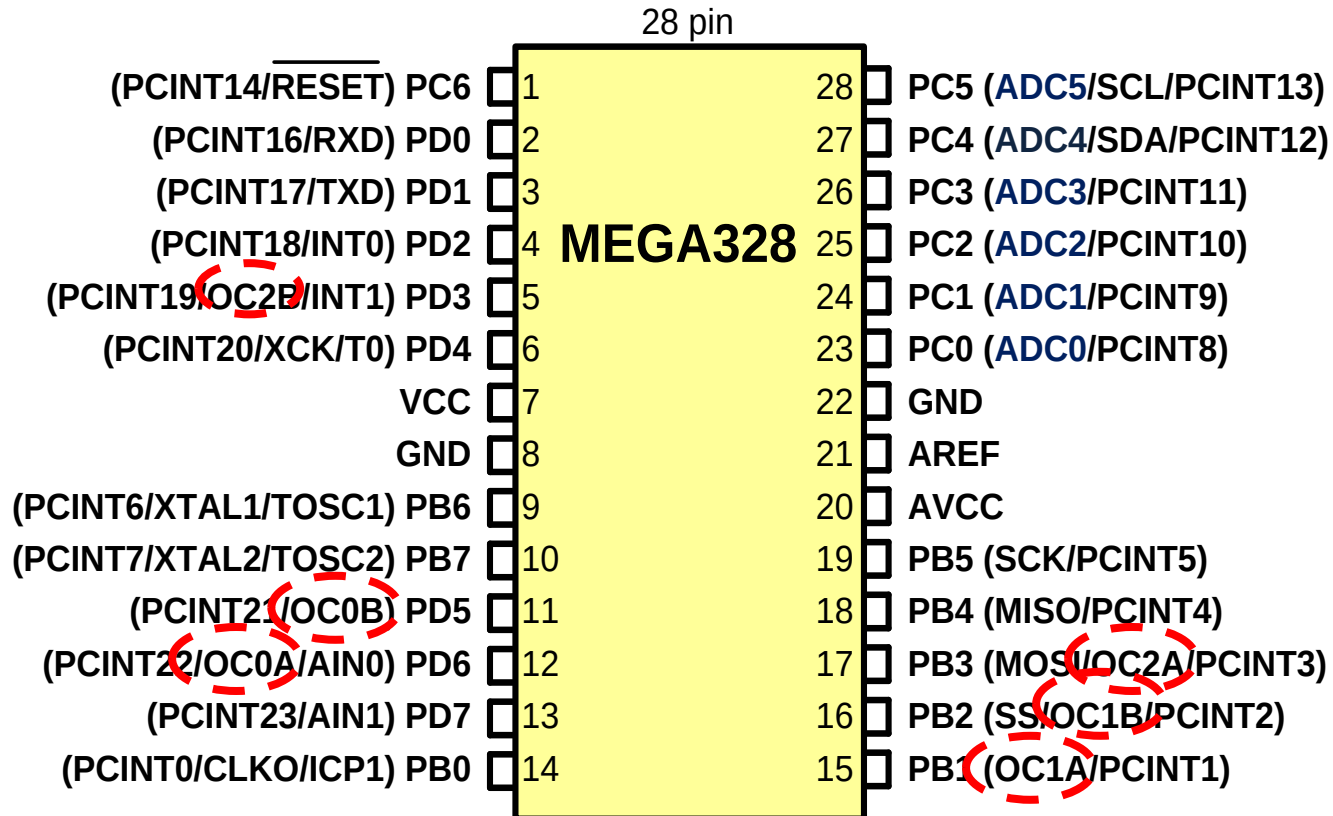
⇒ $\text{duty cycle} = (\text{OCR0A} + 1) * 100 / 256 = 37.5\%$ despejando $\text{OCR0A} = 95$

⇒ si $N=8$ $f_{pwm} = f_{clk} / (256 * N) = 7812.5\text{Hz}$

⇒ Código:

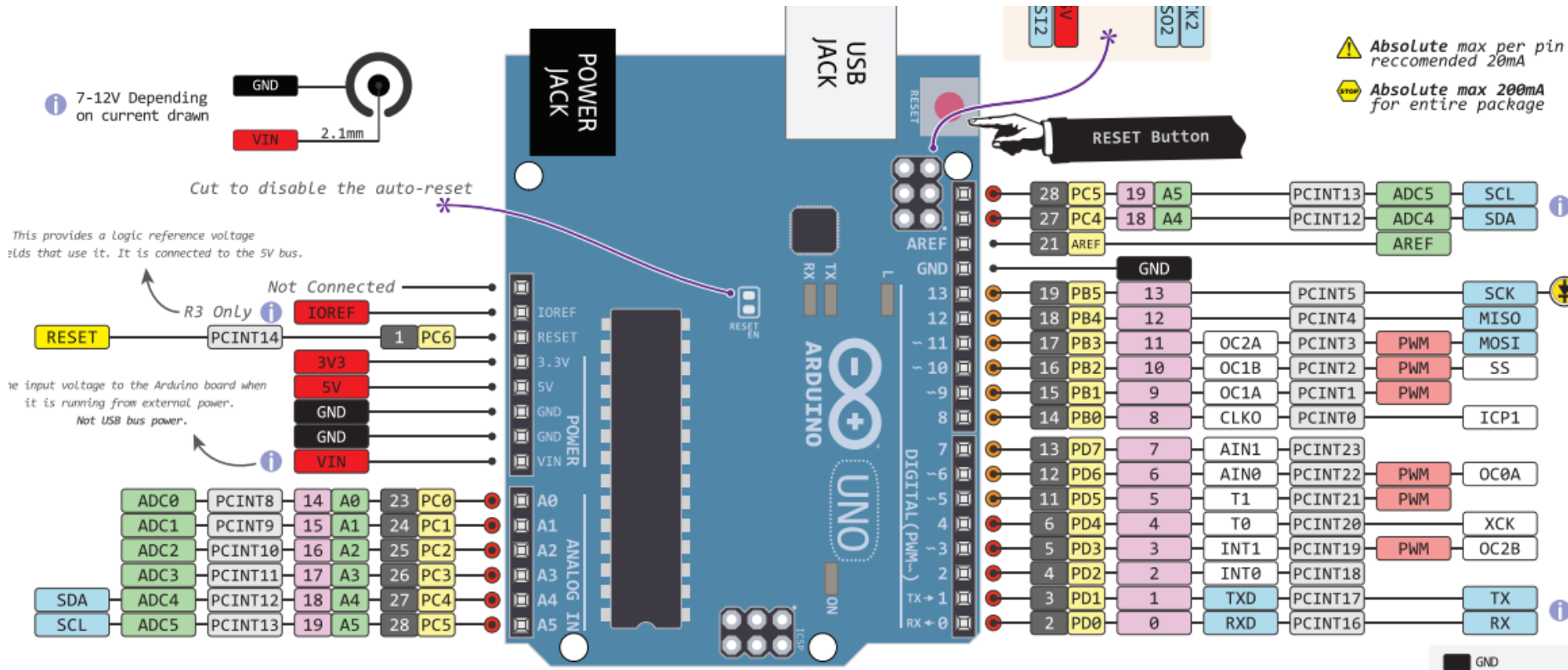
```
DDRD |= (1<<6); //PIN PD6 corresponde a la salida OC0A
OCR0A=95;
TCCR0A=(1<<COM0A1) | (1<<WGM01) | (1<<WGM00); //modo Fast-non inv.
TCCR0B=(1<<CS01); //prediv 8
```

PWM – Atmega328p



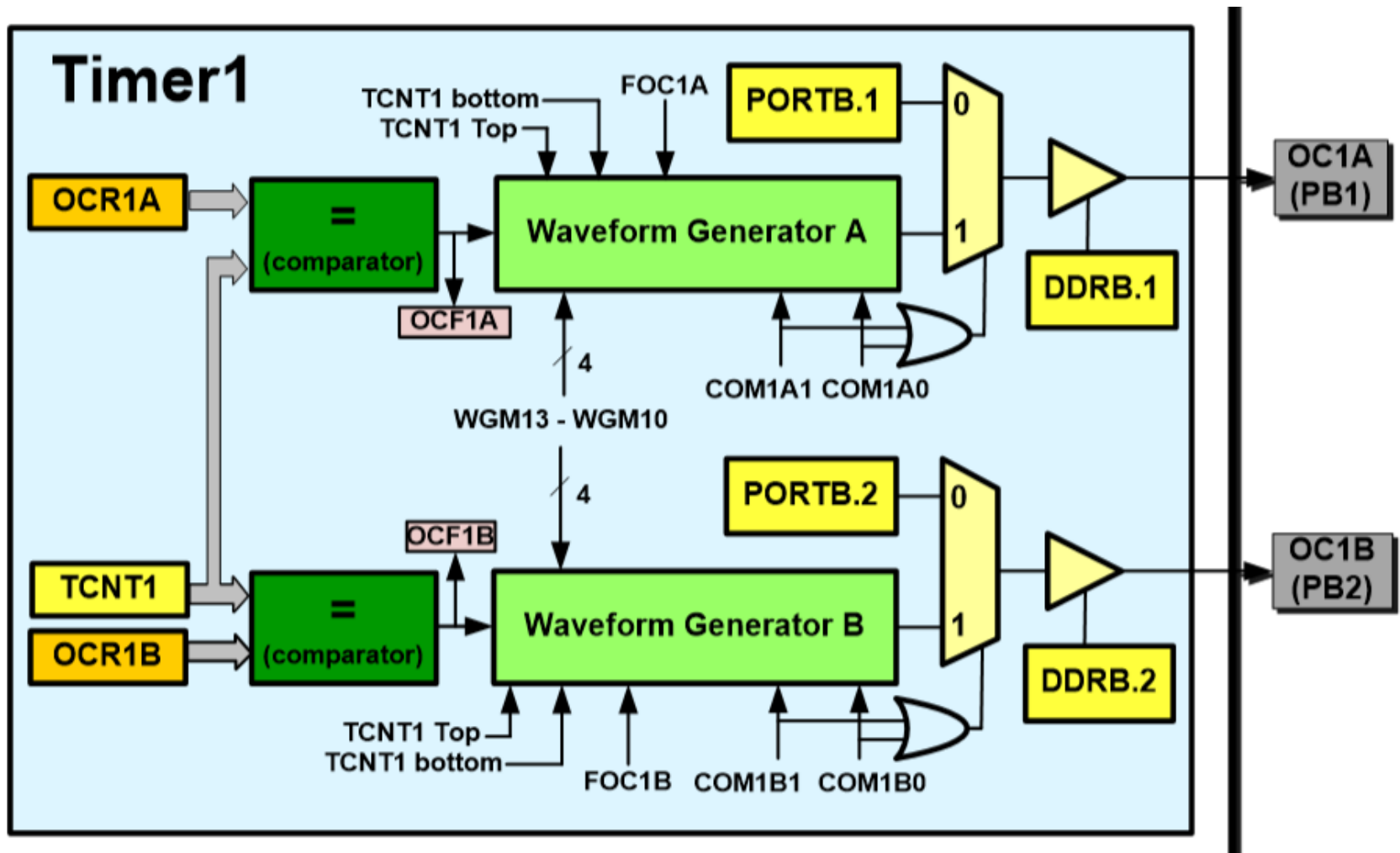
El atmega328p cuenta con 3 pares de salidas PWM

PWM – ARDUINO board



Ojo! En el ecosistema arduino se confunden las salidas PWM con salidas analógicas y se asocian a funciones de biblioteca del tipo *analogWrite()*

PWM – Timer 1 (16 bits)



PWM – Timer 1 (16 bits)

- Modos Fast PWM

Mode	WGM13	WGM12	WGM11	WGM10	Timer/Counter Mode of Operation	Top	Update of OCR1x	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	TOP	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	TOP	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	TOP	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	–	–	–
14	1	1	1	0	Fast PWM	ICR1	TOP	TOP
15	1	1	1	1	Fast PWM	OCR1A	TOP	TOP

PWM – Timer 1 (16 bits)

- 5 Modos Fast PWM dependiendo del valor TOP del contador:

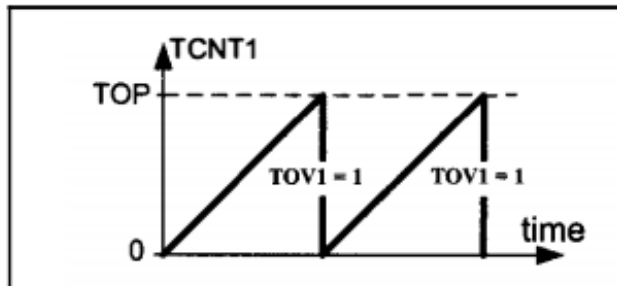


Figure 16-24. Fast PWM Mode

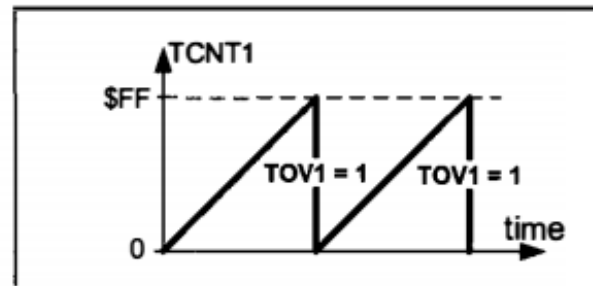


Figure 16-25. TOV in Mode 5

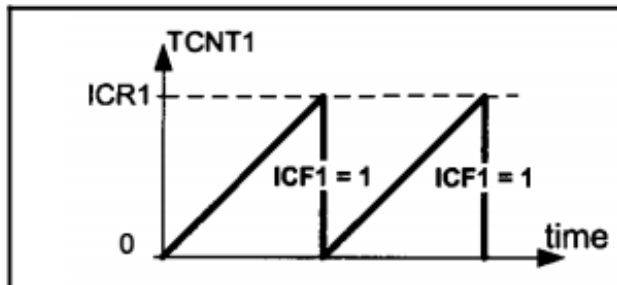


Figure 16-26. Mode 14

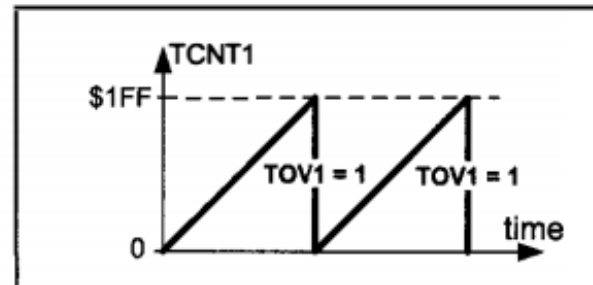


Figure 16-27. TOV in Mode 6

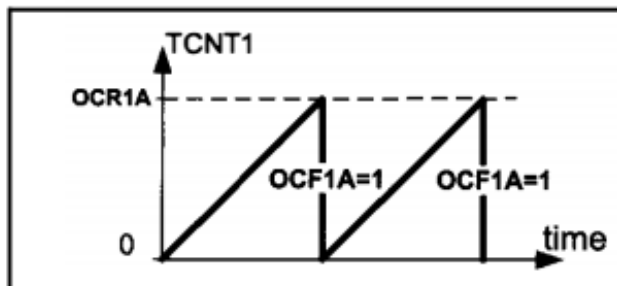


Figure 16-28. Mode 15

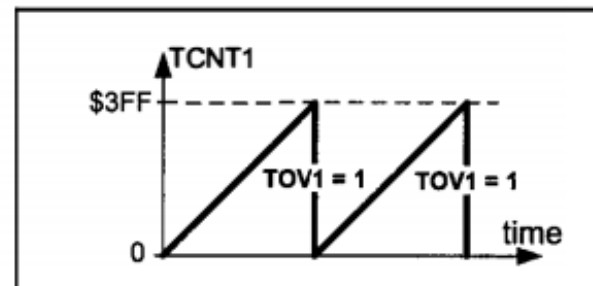


Figure 16-29. TOV in Mode 7

PWM – Timer 1 (16 bits)

- Modo no invertido

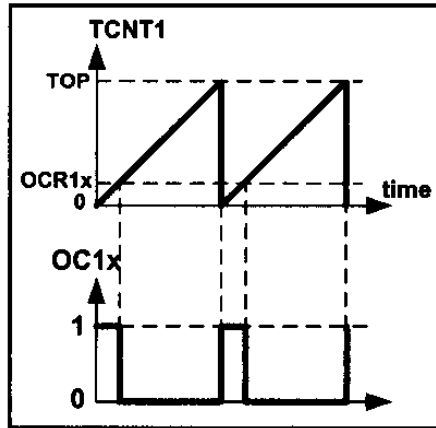


Figure 16-32A. Non-inverted

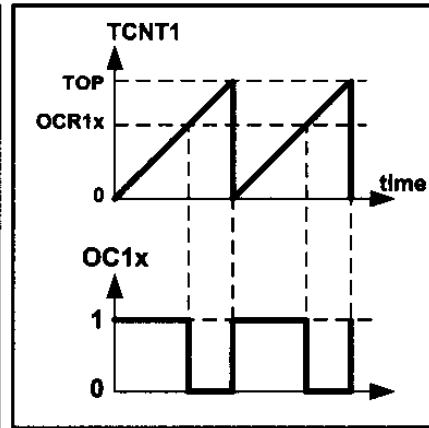


Figure 16-32B. Non-inverted

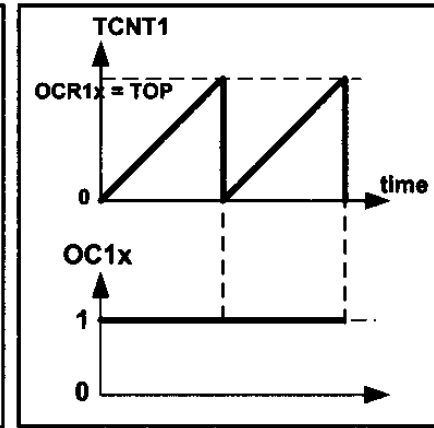


Figure 16-32C. Non-inverted

- Ídem Modo invertido
- Frecuencia fast pwm:

Número de bits de un PWM

$$R_{FPWM} = \frac{\log(TOP + 1)}{\log(2)}$$

$$\left. \begin{aligned} F_{\text{generated wave}} &= \frac{F_{\text{timer clock}}}{Top + 1} \\ F_{\text{timer clock}} &= \frac{F_{\text{oscillator}}}{N} \end{aligned} \right\} \Rightarrow F_{\text{generated wave}} = \frac{F_{\text{oscillator}}}{(Top + 1) \times N}$$

PWM – Timer 1 (16 bits)

- Ciclo de trabajo: $p = \delta/T$

No invertido:

$$\text{Duty Cycle} = \frac{\text{OCR1x} + 1}{\text{Top} + 1} \times 100$$

Invertido:

$$\text{Duty Cycle} = \frac{\text{Top} - \text{OCR1x}}{\text{Top} + 1} \times 100$$

Ejemplo: para ciclo de trabajo 75%, el OCR1A debe ser:

- 191 (en modo 5, no inv)
- 383 (en modo 6, no inv)
- 767 (en modo 7, no inv)

PWM – Timer 1 (16 bits)

- Ejemplo: Generar PWM no inv. M5, $f=31.250\text{Hz}$ y $\text{cicloT}=75\%$.
 - Sup $f_{\text{clk}}=8\text{MHz}$

```
DDRB |= (1<<PB1);    //PIN PB1 corresponde a la salida OC1A
OCR1A=191;
TCCR1A=(1<<COM1A1) | (1<<WGM10); //modo 5 Fast-non inv.
TCCR1B=(1<<CS10) | (1<<WGM12);    //prediv 1
```

- Notar que las posibles frecuencias a generar dependen del prescalador

Prescaler	1	1:8	1:64	1:256	1:1024
Mode = 5	$\frac{F_{\text{oscillator}}}{1 \times 256}$	$\frac{F_{\text{oscillator}}}{8 \times 256}$	$\frac{F_{\text{oscillator}}}{64 \times 256}$	$\frac{F_{\text{oscillator}}}{256 \times 256}$	$\frac{F_{\text{oscillator}}}{1024 \times 256}$
Mode = 6	$\frac{F_{\text{oscillator}}}{1 \times 512}$	$\frac{F_{\text{oscillator}}}{8 \times 512}$	$\frac{F_{\text{oscillator}}}{64 \times 512}$	$\frac{F_{\text{oscillator}}}{256 \times 512}$	$\frac{F_{\text{oscillator}}}{1024 \times 512}$
Mode = 7	$\frac{F_{\text{oscillator}}}{1 \times 1024}$	$\frac{F_{\text{oscillator}}}{8 \times 1024}$	$\frac{F_{\text{oscillator}}}{64 \times 1024}$	$\frac{F_{\text{oscillator}}}{256 \times 1024}$	$\frac{F_{\text{oscillator}}}{1024 \times 1024}$

PWM – Timer 1 (16 bits)

- Para generar otras frecuencias se utilizan los modos 14 o 15 donde TOP se especifica con los registros ICR1 o OCR1A respectivamente.
- Por ejemplo: generar fast pwm, modo 14, $f=80\text{kHz}$

$$80\text{kHz}=8\text{MHz} / N*(\text{TOP}+1) \Rightarrow / N*(\text{TOP}+1) = 8\text{MHz}/80\text{kHz} = 100$$

$$\text{Si } N=1 \text{ (CS1[2:0]=001)}$$

$$\text{TOP}+1=100$$

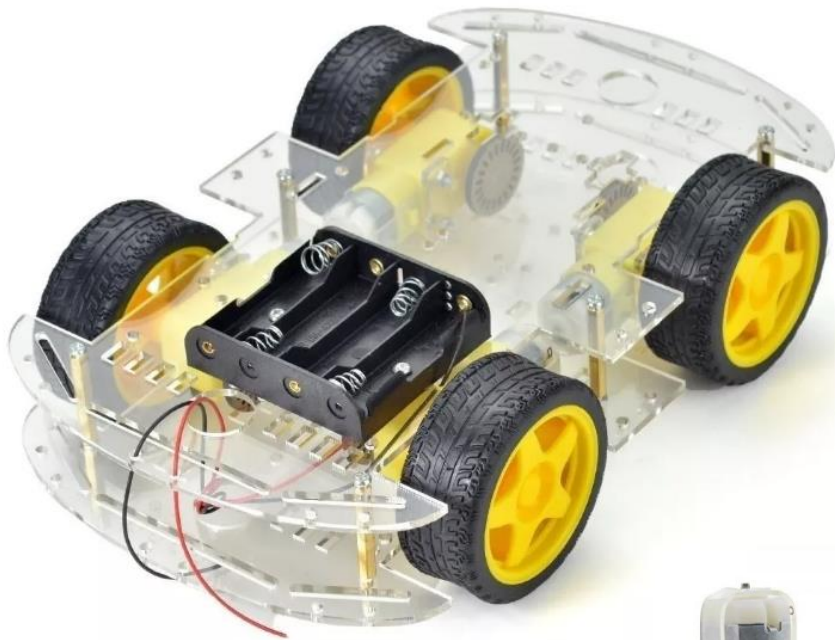
$$\text{TOP}=99$$

- Para ciclo de trabajo 20%

$$20=(\text{OCR1x}+1)*100 / (99+1) \Rightarrow \text{OCR1x}+1=20 \Rightarrow \text{OCR1x} = 19$$

```
DDRB |= (1<<PB1);    //PIN PB1 corresponde a la salida OC1A
ICR1A=99;
OCR1A=19;
TCCR1A=(1<<COM1A1) | (1<<WGM11); //modo 14 Fast-non inv.
TCCR1B=(1<<CS10) | (1<<WGM13) | (1<<WGM12); //prediv 1
```

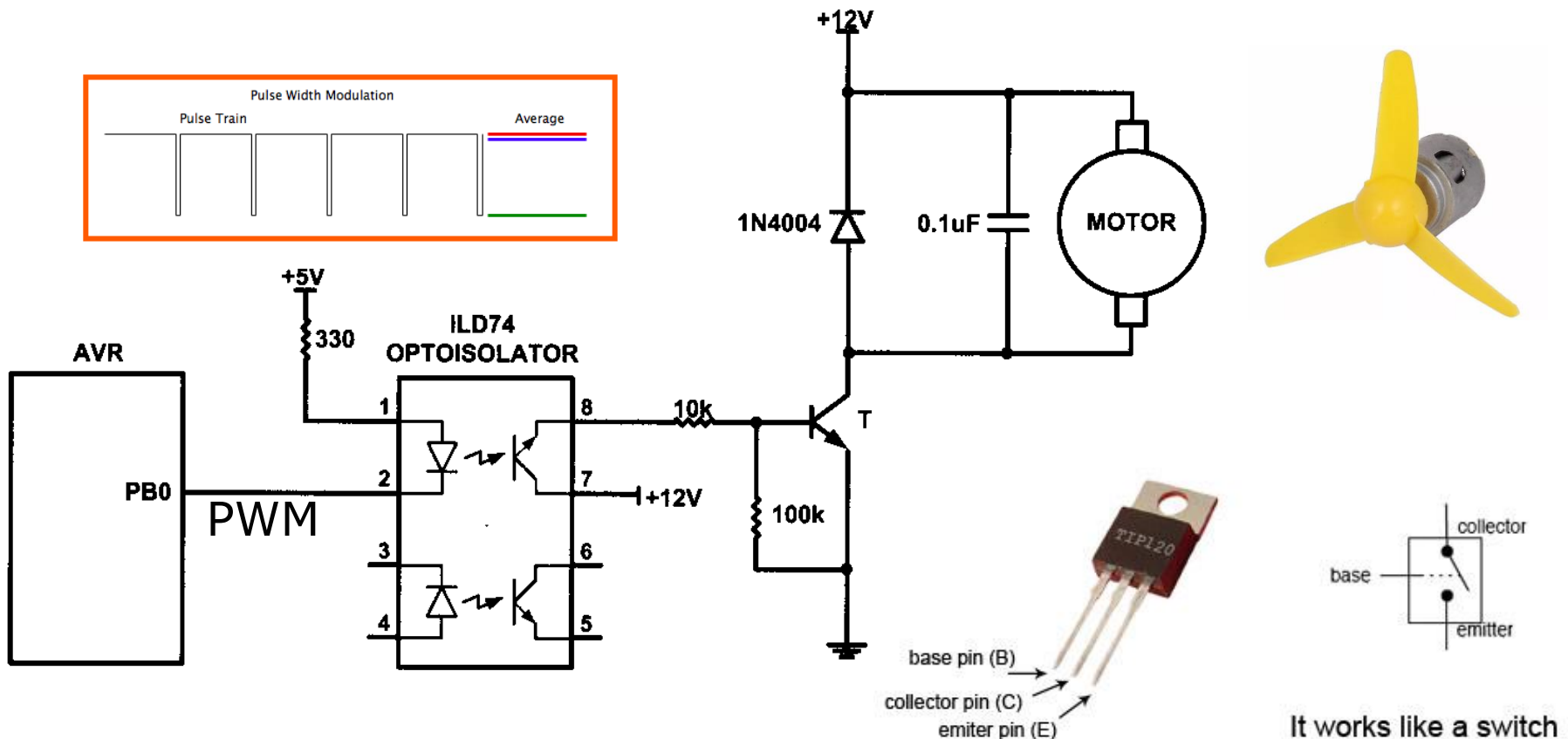
Aplicaciones PWM – control de Motor DC



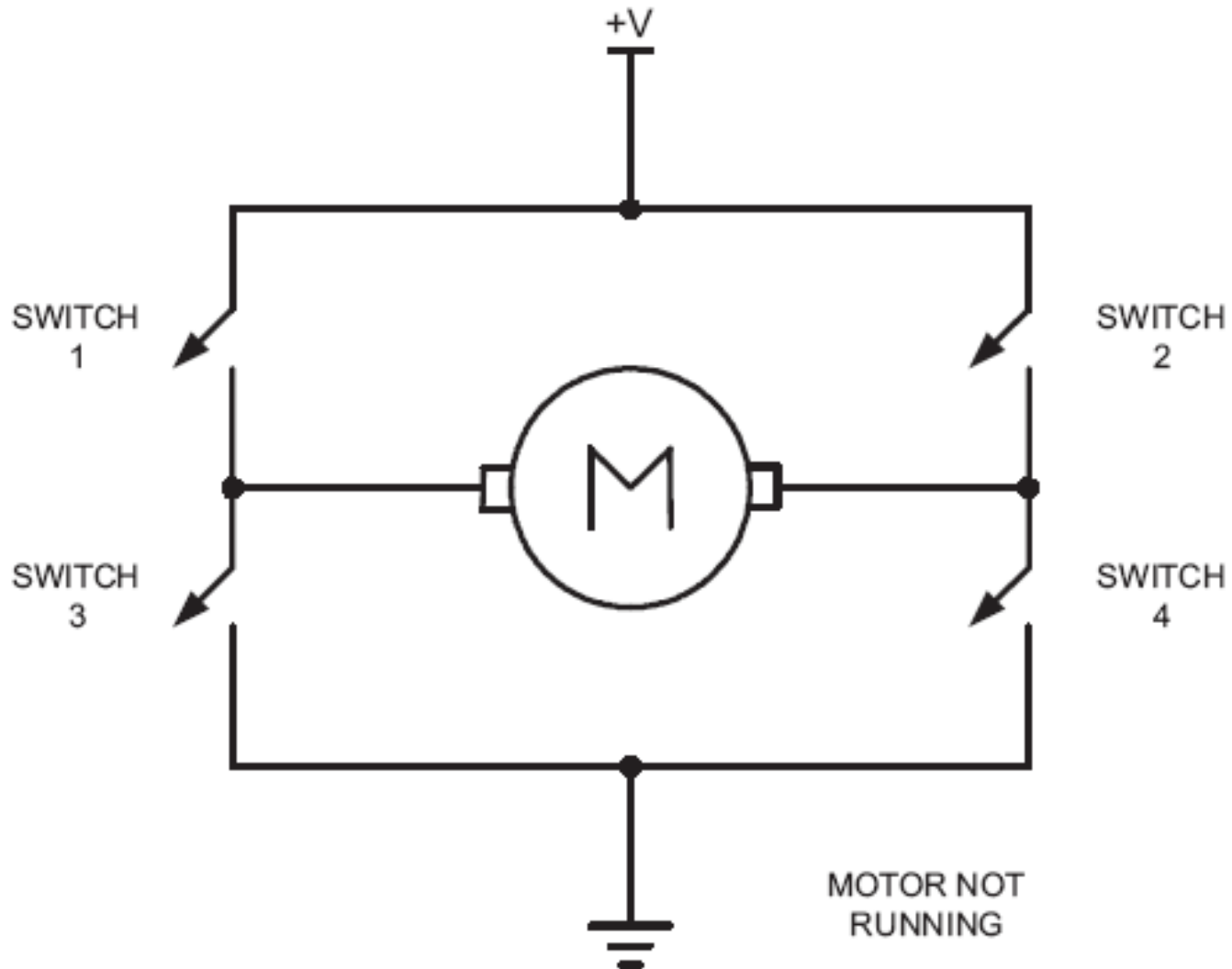
PWM – control de Motor DC

- Ejemplo de aplicación: control de velocidad unidireccional

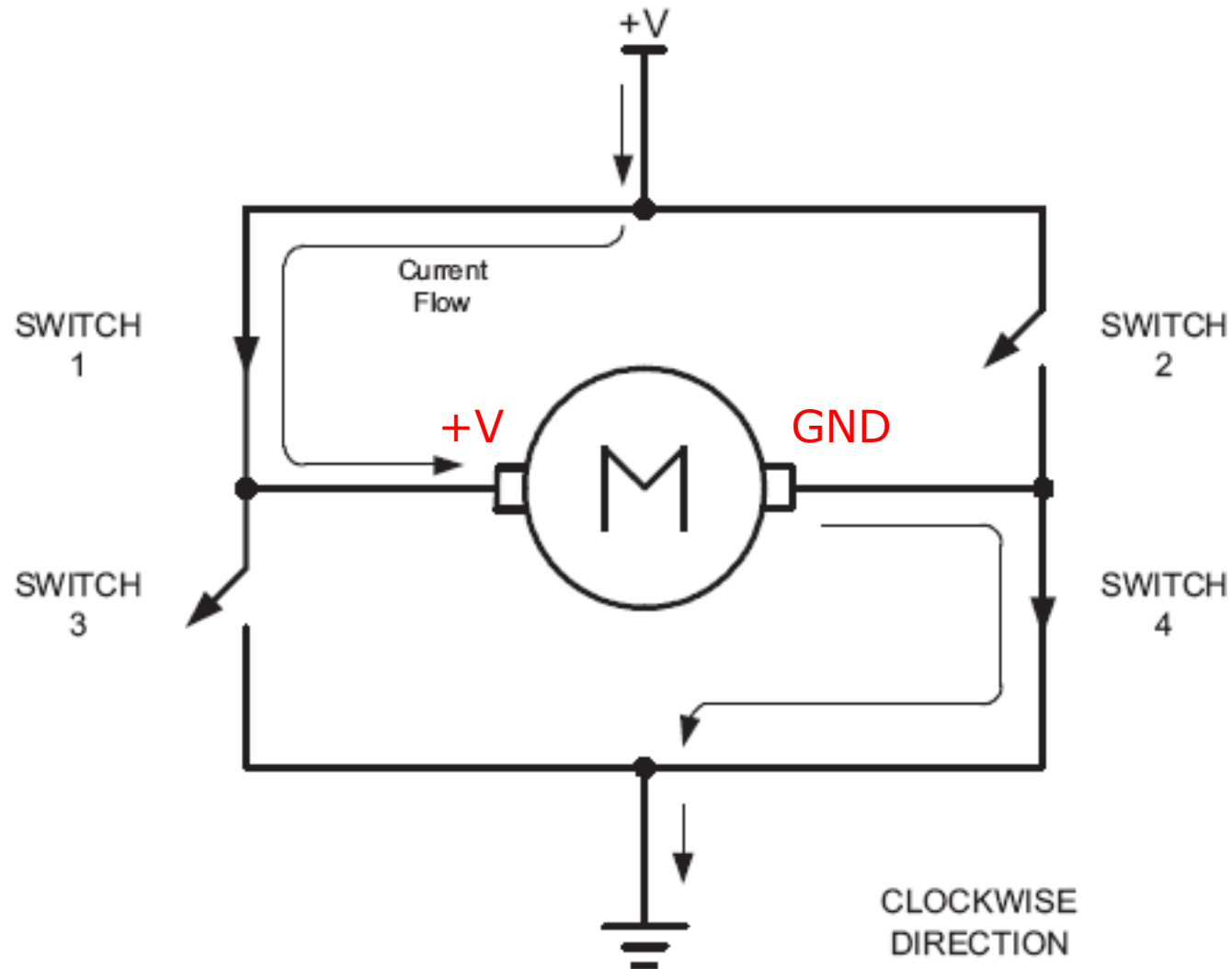
La velocidad de giro del motor se controla con la señal digital aplicada en bornes del motor



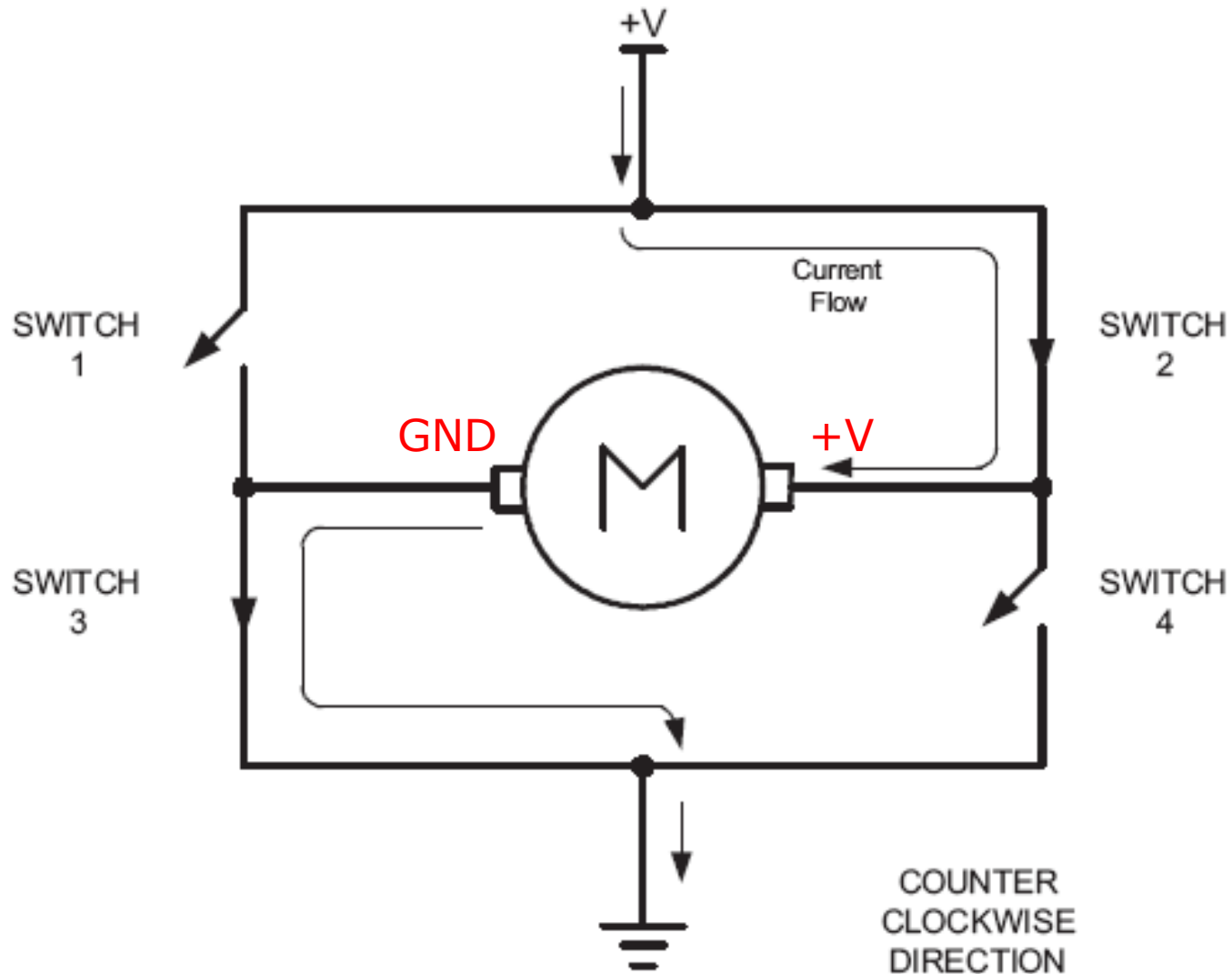
Motor DC control bidireccional



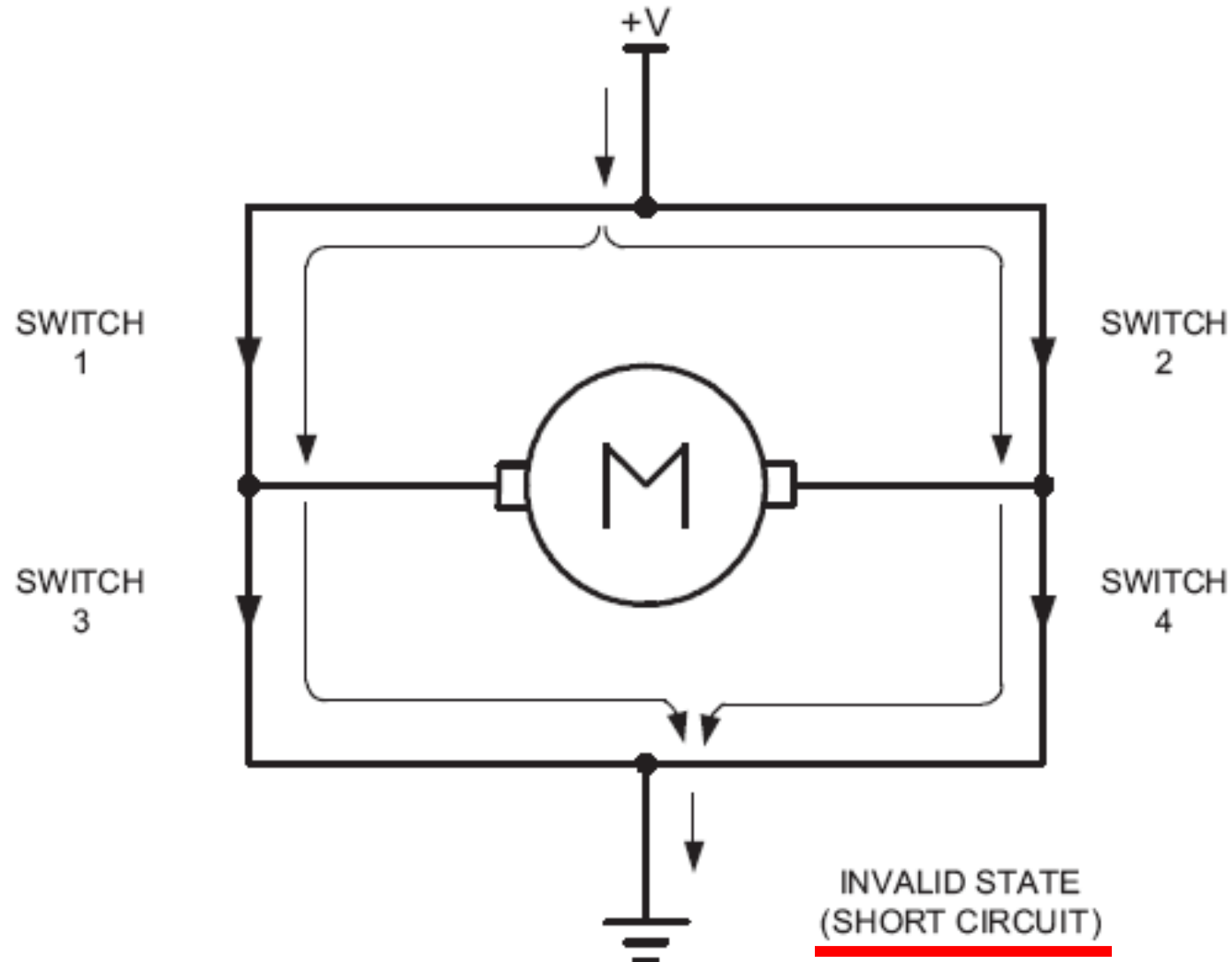
Motor DC control bidirectional



Motor DC control bidirectional



Motor DC control bidirectional (Invalid state)

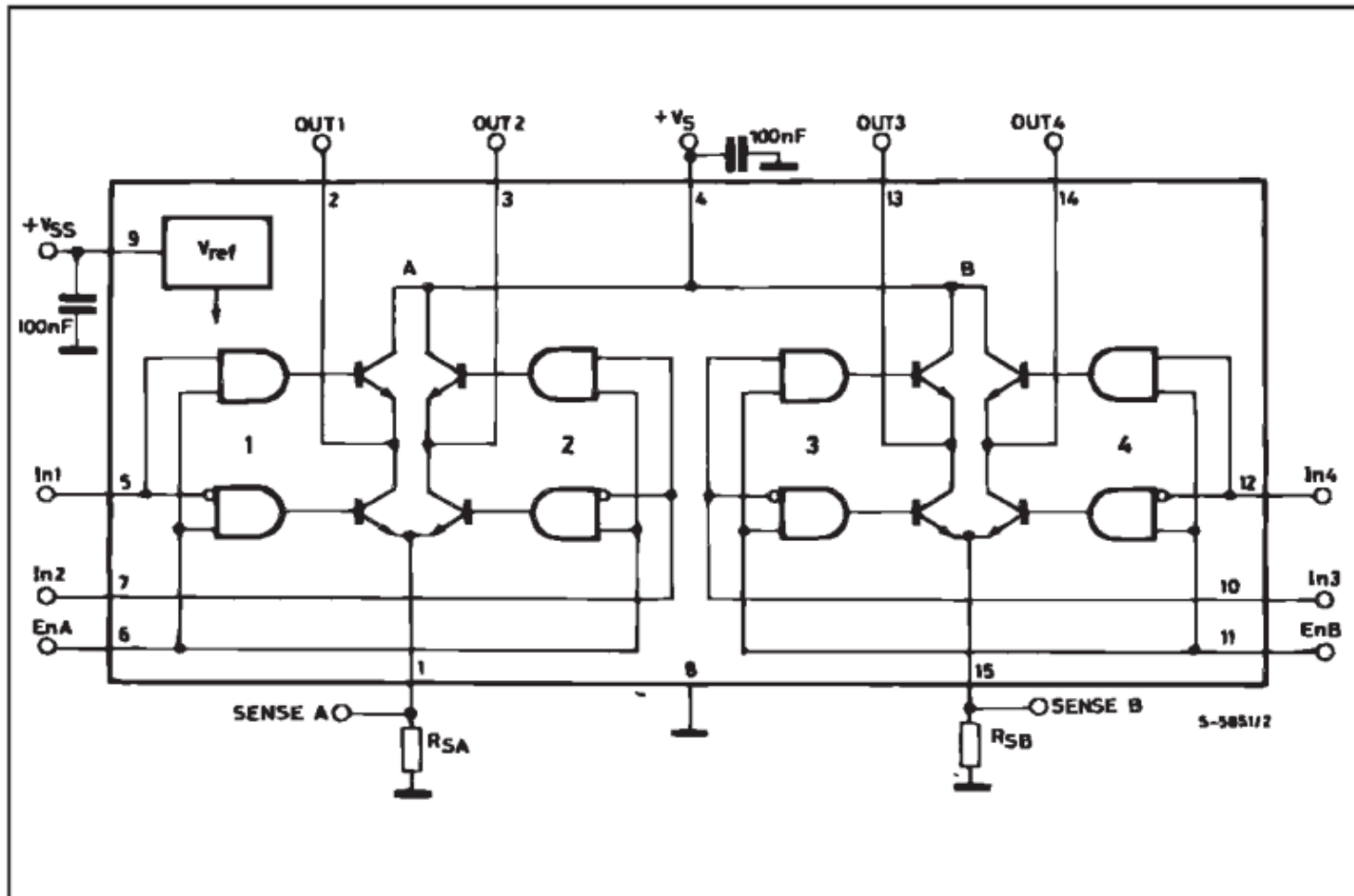


PWM – Motor DC

- Control bidireccional con L298
- Posee 2 puentes H

L298N Motor Controller Truth Table

ENA	IN1	IN2	Description
0	N/A	N/A	Motor A is off
1	0	0	Motor A is stopped (brakes)
1	0	1	Motor A is on and turning backwards
1	1	0	Motor A is on and turning forwards
1	1	1	Motor A is stopped (brakes)

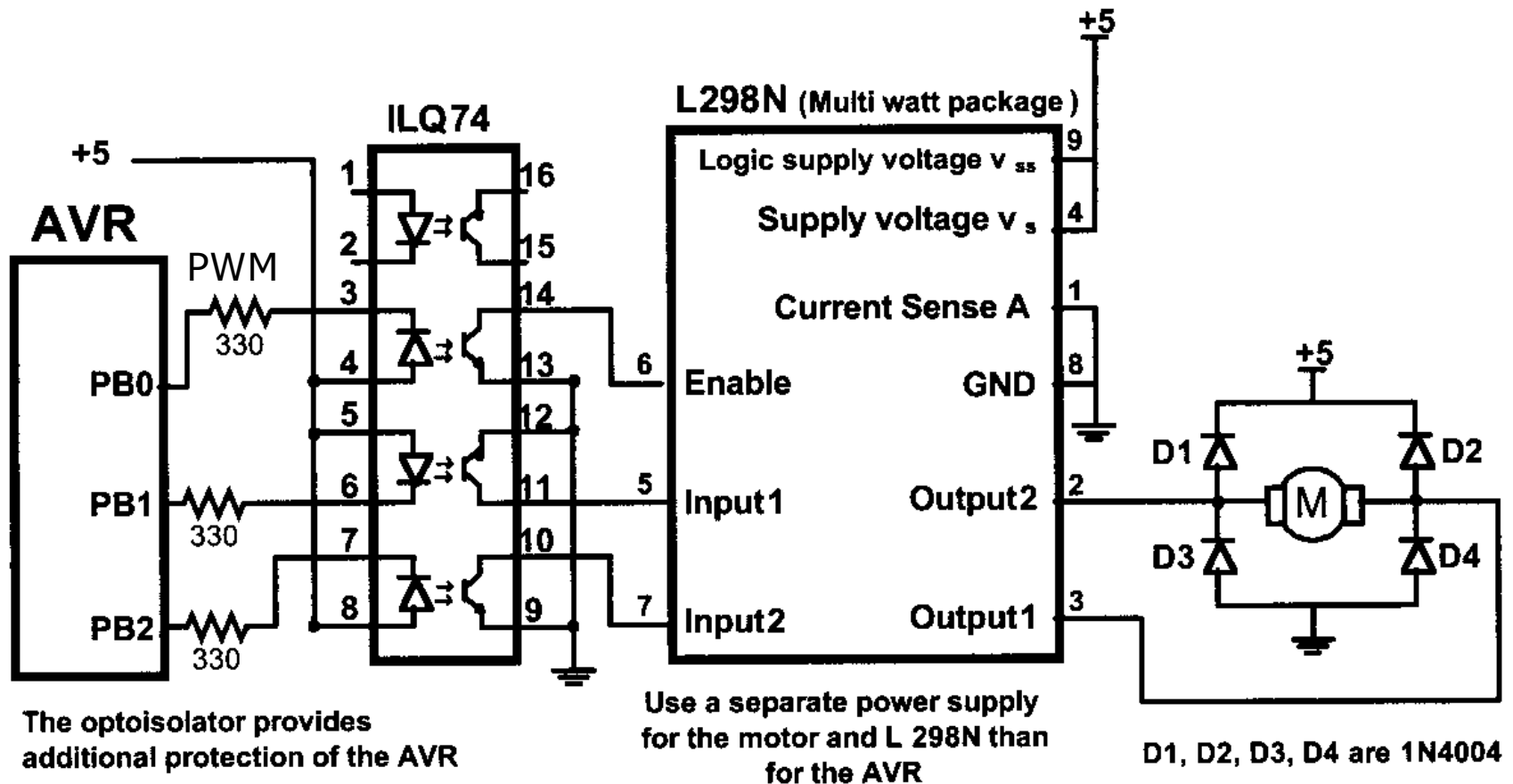


PWM – Motor DC

- Control bidireccional con L298

L298N Motor Controller Truth Table

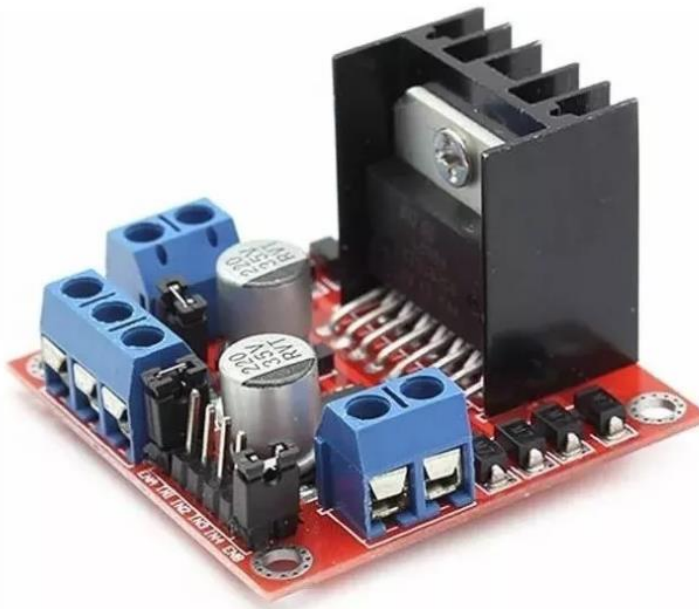
ENA	IN1	IN2	Description
0	N/A	N/A	Motor A is off
1	0	0	Motor A is stopped (brakes)
1	0	1	Motor A is on and turning backwards
1	1	0	Motor A is on and turning forwards
1	1	1	Motor A is stopped (brakes)



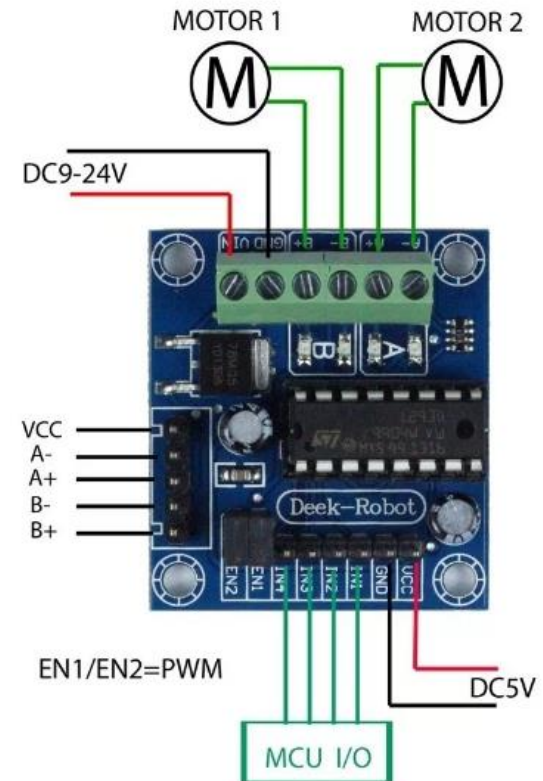
- PB1 y PB2 controlan la dirección de giro (Input 1 y 2) y PB0 controla la velocidad por PWM (Enable)

PWM – Motor DC

- Algunos ejemplos de módulos económicos



L298 (3A)



L293D (1A)

PWM Fase correcta– Timer 0

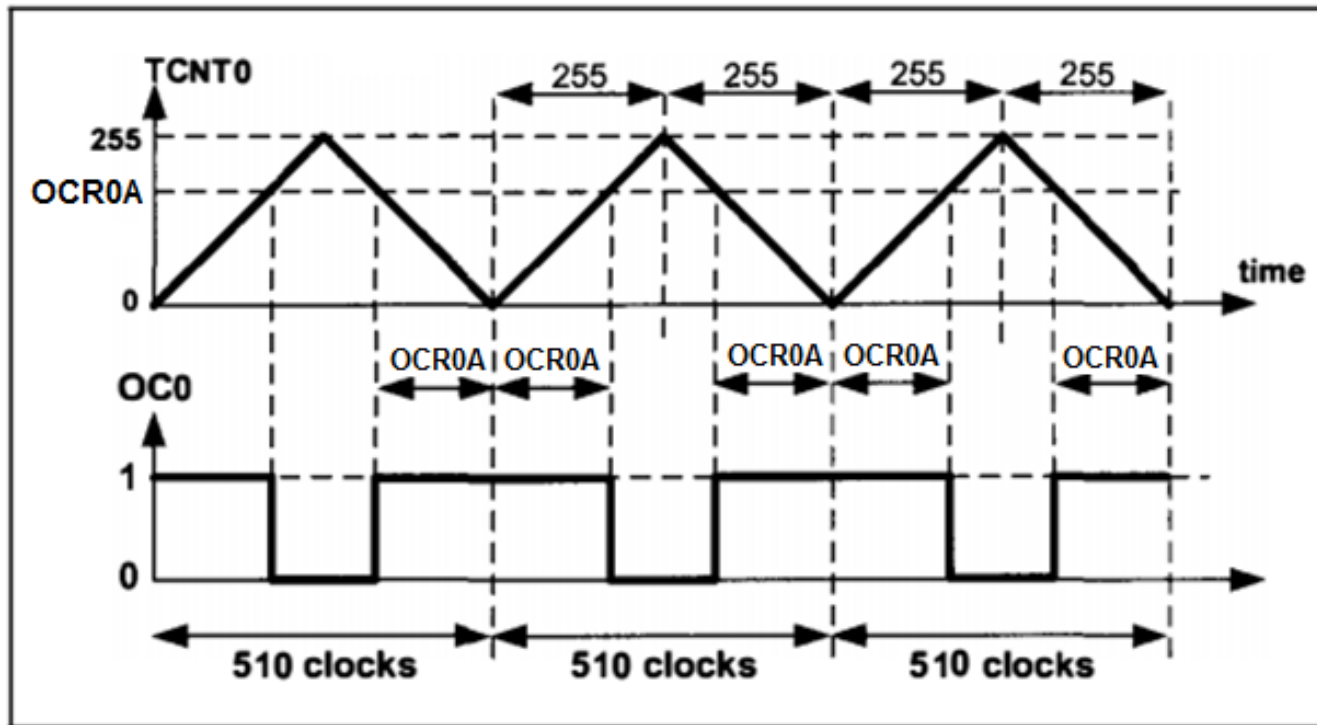
Table 14-8. Waveform Generation Mode Bit Description

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

- Notes: 1. MAX = 0xFF
2. BOTTOM = 0x00

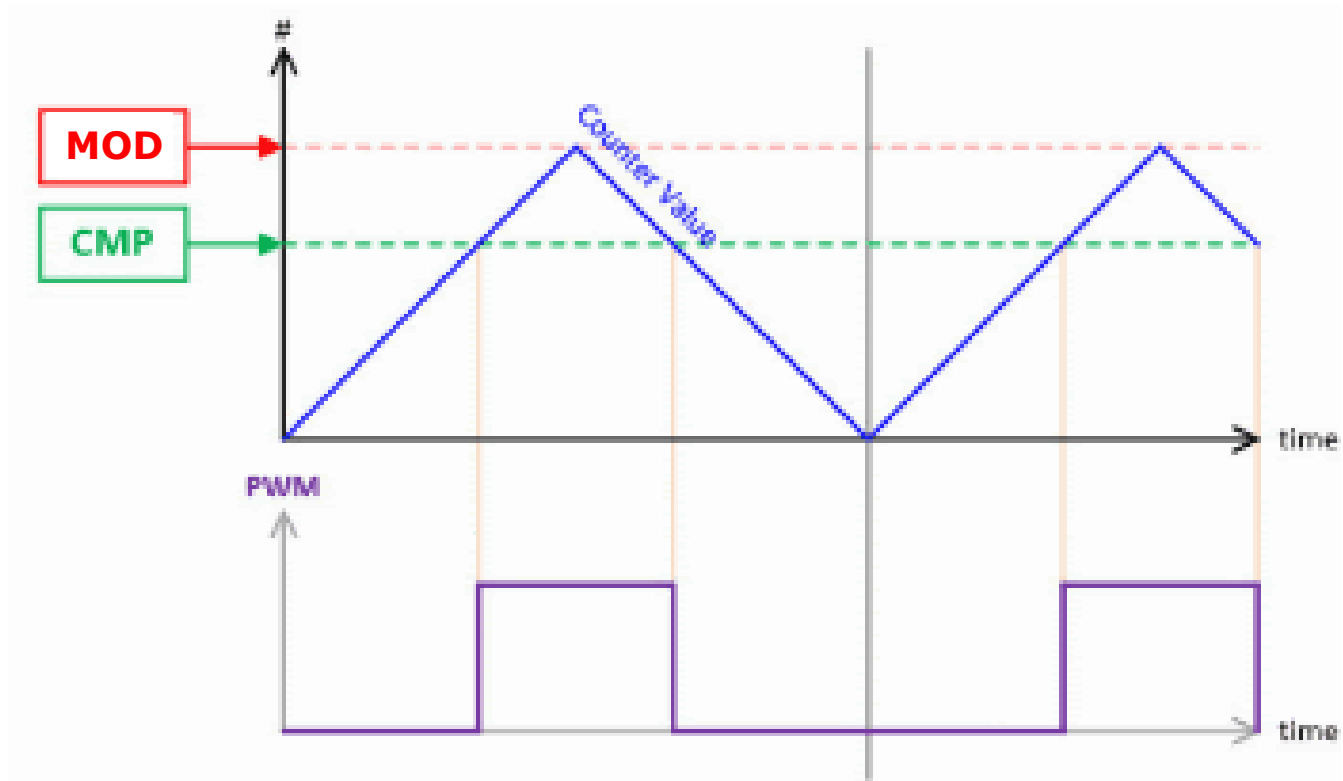
PWM Fase correcta– Timer 0

- Modo Fase correcta



$$\begin{aligned}
 F_{\text{generated wave}} &= \frac{F_{\text{timer clock}}}{510} \\
 F_{\text{timer clock}} &= \frac{F_{\text{oscillator}}}{N}
 \end{aligned}
 \left. \vphantom{\begin{aligned} F_{\text{generated wave}} &= \frac{F_{\text{timer clock}}}{510} \\ F_{\text{timer clock}} &= \frac{F_{\text{oscillator}}}{N} \end{aligned}} \right\} \Rightarrow F_{\text{generated wave}} = \frac{F_{\text{oscillator}}}{510 \times N}$$

PWM Fase correcta– Timer 0



PWM Fase correcta– Timer 1 (16 bits)

- Modos PWM

Mode	WGM13	WGM12	WGM11	WGM10	Timer/Counter Mode of Operation	Top	Update of OCR1x	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	TOP	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	TOP	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	TOP	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	–	–	–
14	1	1	1	0	Fast PWM	ICR1	TOP	TOP
15	1	1	1	1	Fast PWM	OCR1A	TOP	TOP

PWM Fase correcta – Timer 1 (16 bits)

- Modo Fase correcta

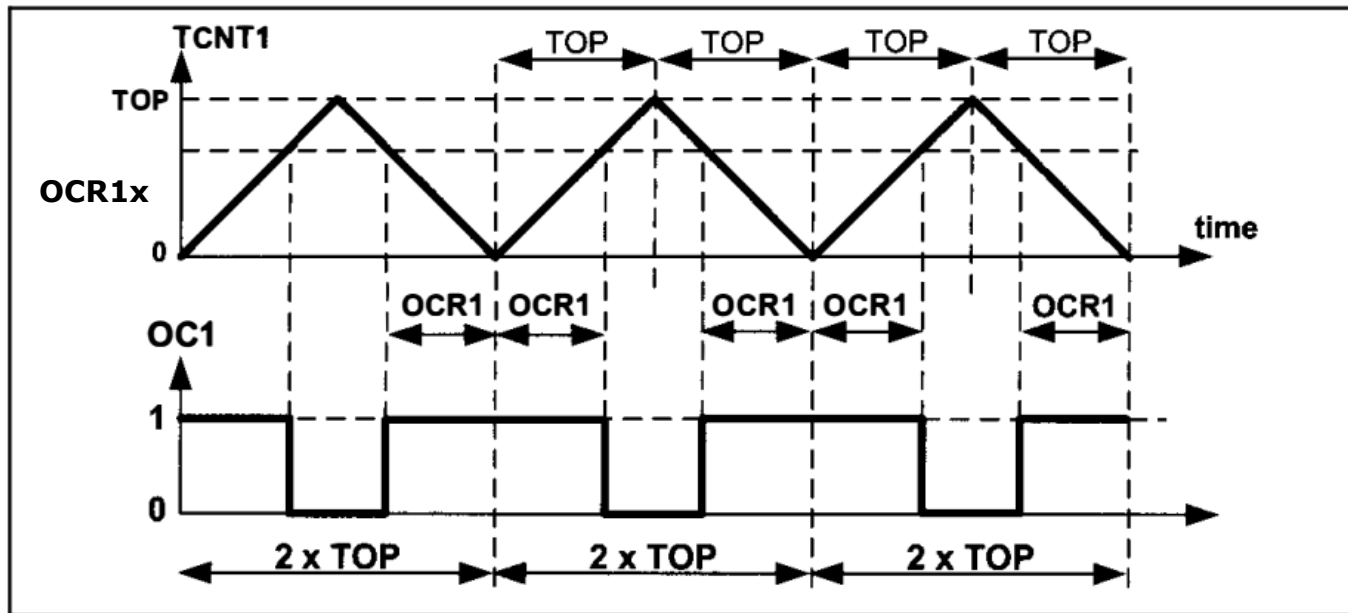


Figure 16-46. Timer/Counter 1 Phase Correct PWM Mode

$$\left. \begin{aligned} F_{\text{generated wave}} &= \frac{F_{\text{timer clock}}}{2 \times \text{Top}} \\ F_{\text{timer clock}} &= \frac{F_{\text{oscillator}}}{N} \end{aligned} \right\} \Rightarrow F_{\text{generated wave}} = \frac{F_{\text{oscillator}}}{2 \times N \times \text{Top}}$$

PWM Fase correcta – Timer 1 (16 bits)

- Modo Fase correcta

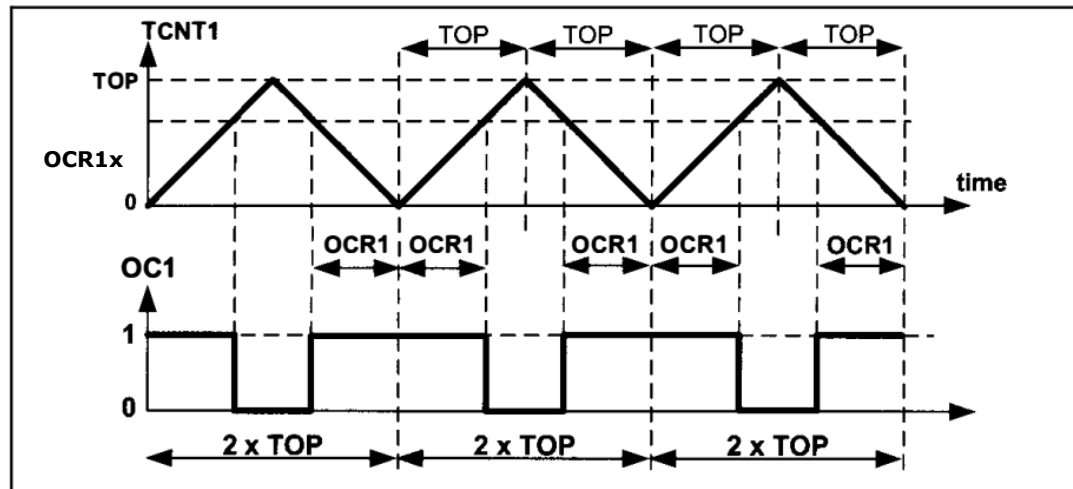


Figure 16-46. Timer/Counter 1 Phase Correct PWM Mode

- No Invertido:

$$\text{Duty Cycle} = \frac{2 \times \text{OCR1A}}{2 \times \text{Top}} \times 100 \quad \Rightarrow \quad \text{Duty Cycle} = \frac{\text{OCR1A}}{\text{Top}} \times 100$$

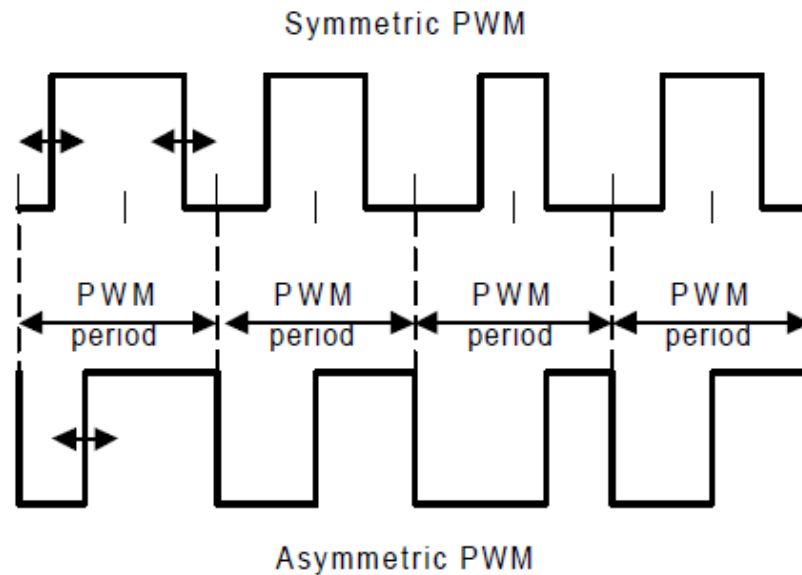
- Invertido:

$$\text{Duty Cycle} = \frac{2 \times \text{Top} - 2 \times \text{OCR1A}}{2 \times \text{Top}} \times 100 \quad \Rightarrow \quad \text{Duty Cycle} = \frac{\text{Top} - \text{OCR1A}}{\text{Top}} \times 100$$

Notar que puede obtenerse el rango 0 a 100% en un mismo modo

PWM Fase correcta – Timer 1 (16 bits)

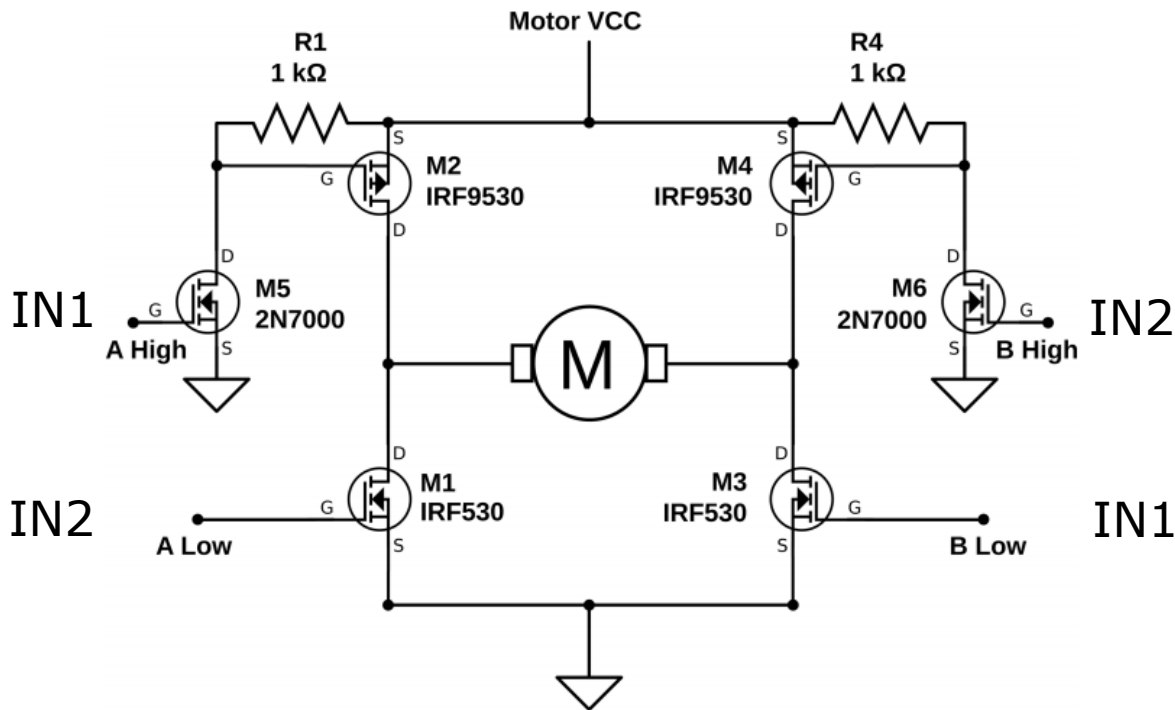
- Diferencias entre modos:



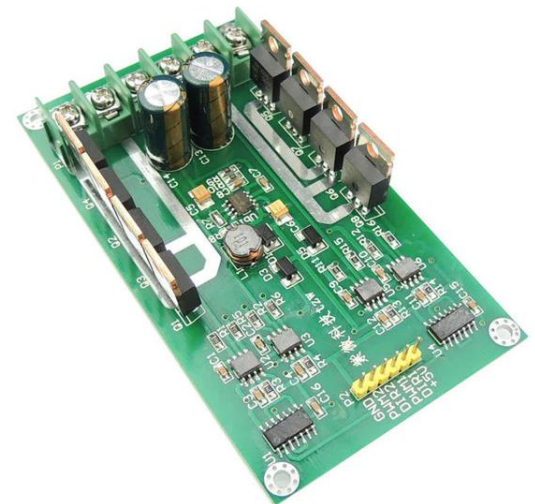
- Modo simétrico se utiliza preferentemente para el control de motores de múltiples fases como los motores de inducción o los BLDC (generalmente 3 o 6 fases).
- En fase correcta, el centro del pulso no varía, con lo cual puede utilizarse como instante para sensar la corriente.
- En motores de más de una fase el contenido armónico generado es menor.

PWM Fase correcta– Motor DC

- Ejemplo de aplicación: Control Bidireccional con Puente H
 - Cuando el uso de chips puente H no es posible por el alto consumo del motor, es necesario armar el circuito con transistores de potencia discretos.



Las señales IN1 e IN2
No se pueden activar
simultáneamente.
Se requiere de un zona de guarda
antes de la transición (Dead Zone)



PWM Fase correcta– Timer 1 (16 bits)

- En modo Fase correcta, la zona muerta de las señales PWM diferenciales puede controlarse fácilmente desplazando los valores de los registros:

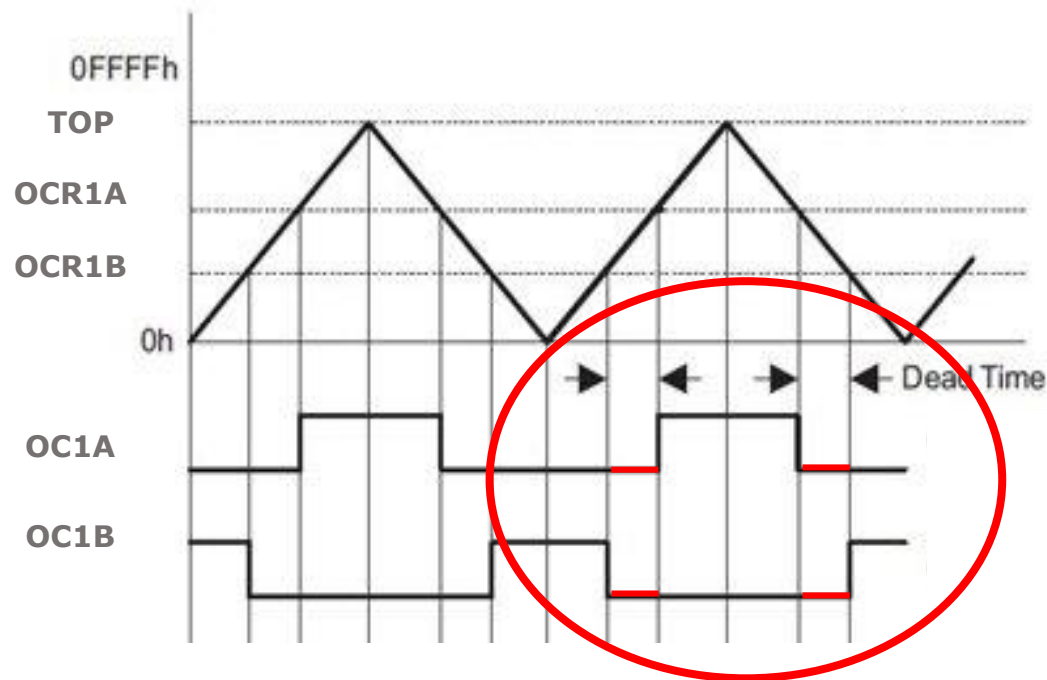
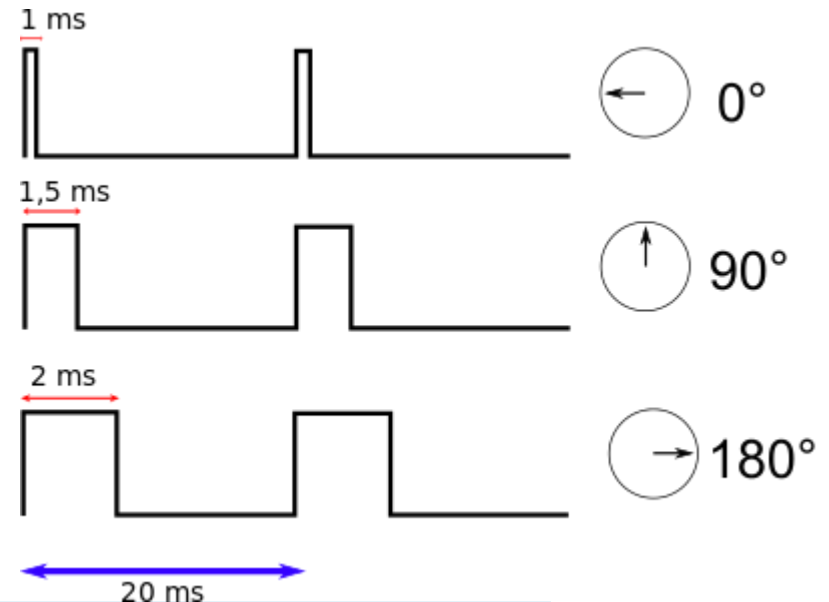
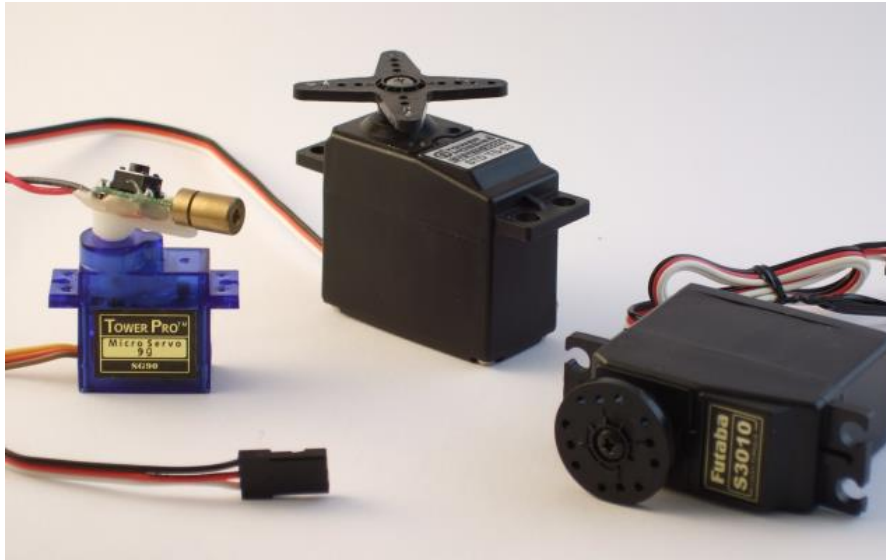


Figure 12-9. Output Unit in Up/Down Mode

OTRAS APLICACIONES

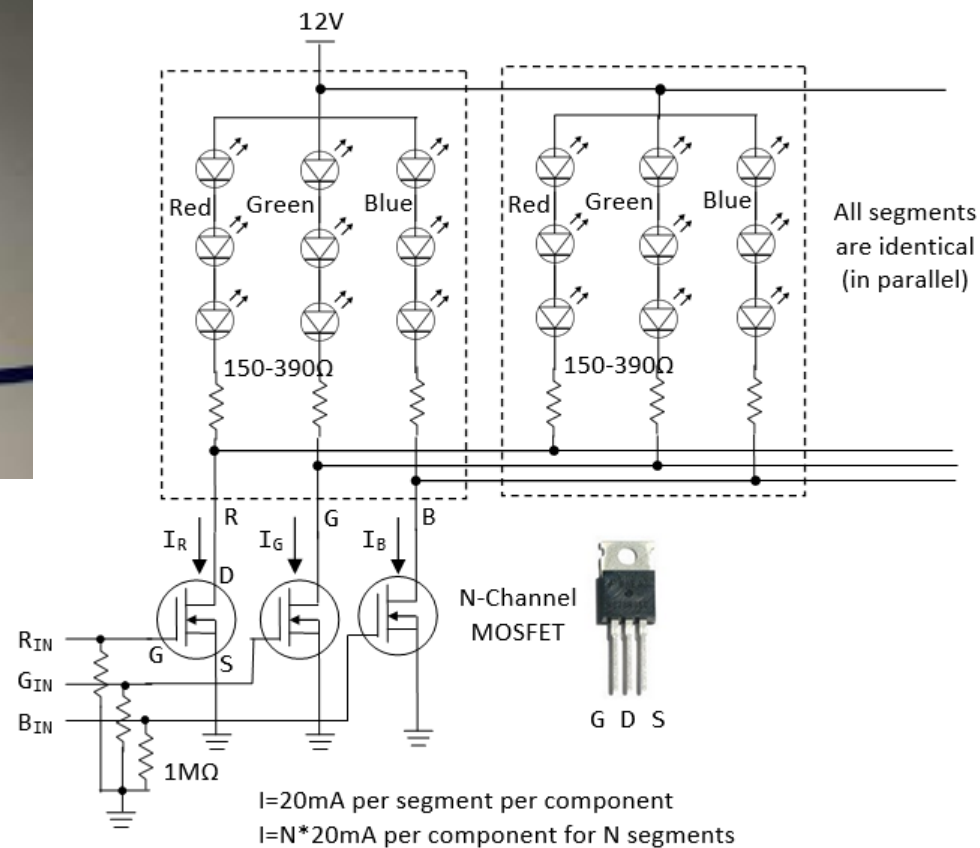
<https://controlautomaticoeducacion.com/arduino/servomotor/>



```
static inline void initTimer1Servo(void) {  
    /* Set up Timer1 (16bit) to give a pulse every 20ms */  
    /* Use Fast PWM mode, counter max in ICR1 */  
  
    TCCR1A |= (1 << WGM11);  
    TCCR1B |= (1 << WGM12) | (1 << WGM13);  
    TCCR1B |= (1 << CS10); /* /1 prescaling -- counting in microseconds */  
    ICR1 = 20000; /* TOP value = 20ms */  
    TCCR1A |= (1 << COM1A1); /* Direct output on PB1 / OC1A */  
    DDRB |= (1 << PB1); /* set pin for output */  
}  
  
OCR1A = PULSE_MID;  
_delay_ms(1500);
```

OTRAS APLICACIONES

Circuito de control de Tiras de LEDs RGB



Bibliografía:

- *The AVR microcontroller & Embedded Systems*. Mazidi, Naimis, CH14,15 y16
- Libro Digital de Felipe Espinosa, CH4
- *DAC by using PWM: Using PWM to generate analog output, AN110*, June Choi, Hynix.
- *Using PWM Output as a Digital-to-Analog Converter on a TMS320F280x Digital Signal Controller, SPRAA88A*, David Alter, Texas Instruments, 2008.
- Web: <https://controlautomaticoeducacion.com/arduino/pwm-arduino/>

